

Multi-Agent Conversational Architectures to Automate Prompt Engineering

Isaac Watson

Academic Supervisor: Prof. Philip Treleavan

Technical Supervisors: Sergio Alvares, Gamal Crichton

Faculty of Engineering

Department of Computer Science

University College London

A Project Report Presented in Partial Fulfillment of the Degree

Computational Statistics and Machine Learning

September 2024

Abstract

This thesis explores the use of multi-agent large language model (LLM) systems to automate prompt engineering, a critical aspect in enhancing LLM performance. Carefully engineered prompts are essential for improving LLM task-specific accuracy and generalisability. The motivation for this research is drawn from the demonstrated effectiveness of conversational LLM agents in performing a wide range of complex tasks by utilising constitutional experts and collaborative feedback mechanisms. Multi-agent LLM systems have been underexplored in prompt optimisation but have shown promising results in other domains. This study evaluates the potential of three different multi-agent architectures - Authoritarian, Market-Based, and Hierarchical - to autonomously refine prompts, thereby improving the robustness and efficiency of LLM outputs across various user and contextual scenarios. It also looks to learn from the actions of the agents to help standardise the field of prompt engineering.

Prompt optimisation is notoriously difficult and time-consuming, especially for complex tasks. Various approaches have been designed to address this, including evolutionary algorithms, reinforcement learning, and gradient-based methods. This thesis investigates the lesser-explored domain of multi-agent systems for prompt optimisation, proposing three specific architectures exemplifying both collaborative and competitive strategies.

The thesis comprises three investigations, each of which the author believes represents an original contribution to the field of automated prompt optimisation.

1. **Authoritarian, Market-Based and Hierarchical Optimisation Frameworks.** This study formalises the multi-agent frameworks and assesses their performance when implemented with two state-of-the-art LLMs on benchmark datasets (code generation, mathematical reasoning, sentiment analysis). The differences in the architectures are highlighted and performance comparatively analysed against zero-shot, zero-shot chain-of-thought, and emotive prompting.
2. **Enhancing Open-Source SLMs to Rival Third-Party Models.** This study critically evaluates the performance of prompts generated by the proposed multi-agent frameworks when utilised by low-compute open-source SLMs (Small Language Models). It investigates the differential impact of the optimisation architectures on language models with varying computational capabilities and examines whether the performance gap between less powerful models and best-in-class models can be narrowed using prompts generated by these frameworks.

3. **Agent Interactions for High-Performing Prompt Generation.** The final investigation delves into the conversational dynamics between agents during the process of optimising a high performing prompt. By analysing conversation logs producing the best performing prompts, this study aims to expose techniques and strategies that agents employ to improve prompts that humans may not.

This thesis presents the following original contributions to science:

1. **Development of domain-adaptable prompt optimisation frameworks.** This research formalises and introduces three distinct multi-agent architectures - Authoritarian, Market-Based, and Hierarchical - designed for domain-adaptable autonomous prompt refinement. The frameworks capitalise on the strengths of conversational LLMs and agent personas to tailor the optimisation process. The frameworks explore both collaborative and competitive strategies, each with their strengths and weaknesses.
2. **Comprehensive evaluation of multi-agent architectures for prompt optimisation.** This contribution provides empirical evidence for the effectiveness of the frameworks versus traditional prompt improvement techniques. The analysis offers critical insights into the conditions under which these optimisation frameworks are most effective, and specifically demonstrates how they can enhance the performance of more accessible smaller models, bringing their capabilities closer to those of state-of-the-art third-party models, thus enhancing cost-effectiveness.
3. **Identification of best practices employed by LLM agents during prompt optimisation.** This research provides a unique perspective on prompt engineering, contributing to the advancement of its standardisation. It provides valuable insights into how different multi-agent architectures pursue prompt improvement, revealing innovative strategies and potentially novel prompt solutions that emerge from knowledge sharing and strategic interactions among agents.

Impact Statement

This thesis makes the following significant contributions to industry:

1. **Enhancement of LLM effectiveness across diverse domains.** This research introduces three innovative frameworks that advance prompt engineering for products utilising LLMs. These frameworks are adaptable across various domains and can be customised to ensure prompts are aligned with specific product requirements. By optimising the prompt generation process, they substantially improve the efficiency and effectiveness of LLM-driven operations.
2. **Increased robustness and reliability in precision-driven, regulated industries.** The research develops frameworks that mitigate the risk of errors in LLM outputs, a critical factor in industries where precision and compliance are paramount. The proposed multi-agent approaches allow for independent validation and cross-checking of prompts by multiple agents, thereby reducing the likelihood of errors or biases in the final output. This contribution is particularly valuable in regulated sectors such as healthcare, finance, and law.
3. **Improved efficiency of LLM use by non-technical users.** Prompts are famously difficult to curate, even for AI experts. This research provides non-technical users, who will undoubtedly soon be required to utilise generative AI, with the prompting ability of experts, contributing to increased productivity and quality of output.
4. **Optimisation of resources in generative AI engineering teams.** The automation of prompt engineering, as facilitated by the frameworks developed in this research, streamlines the prompt generation process, minimising the need for extensive human oversight. This allows engineers to focus on more technically demanding tasks, thereby enhancing overall productivity and cost-effectiveness within generative AI engineering teams.

Acknowledgements

I would like to express my gratitude to my primary supervisor, Prof. Philip Treleavean, for their invaluable guidance, support, and encouragement throughout writing this thesis.

I am also thankful to my industry supervisors at ZILO, Sergio Alavares and Gamal Crichton, for their practical insights, advice, and support, which greatly enhanced the relevance and impact of this work.

I would like to voice my appreciation to ZILO for providing the necessary resources and support that were essential for this research.

This thesis would not have been possible without their combined efforts and mentorship.

Declaration

I, Isaac Watson, declare that the thesis has been composed by myself and that the work has not been submitted for any other degree or professional qualification. I confirm that the work submitted is my own, except where work which has formed part of jointly-authored publications has been included and referenced. The report may be freely copied and distributed provided the source is explicitly acknowledged.

I. Watson

09/09/24

Signature

Date

Table of Contents

List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Research Motivation	1
1.2 Research Objectives	3
1.3 Research Experiments	3
1.4 Scientific Contributions	5
1.5 Thesis Structure	5
2 Background and Literature Review	7
2.1 Emergence of LLMs	7
2.2 Prompt Engineering	8
2.3 Automated Prompt Optimisation	9
2.4 Multi-Agent Systems	11
3 Authoritarian, Market-Based and Hierarchical Optimisation Frameworks	14
3.1 Introduction	14
3.2 Background Literature	16
3.3 Methodology	17
3.3.1 Formalisation of Multi-Agent Architectures	17
3.3.2 Benchmark Datasets and Evaluation	20
3.3.3 Experimental Procedure	21
3.4 Results	22
3.4.1 Qualitative Analysis of Prompts	22
3.4.2 Quantitative Analysis of Prompts	23
3.4.3 Latency of Frameworks	25
3.5 Discussion	27
4 Enhancing Open-Source SLMs to Rival Third-Party Models	31
4.1 Background Literature	32
4.2 Methodology	33

4.2.1	Evaluating Locally Hosted Models	34
4.2.2	Experimental Procedure	34
4.3	Results	34
4.3.1	Mistral-V0.3-7B	34
4.3.2	Llama-3.1-8B	37
4.4	Discussion	39
5	Agent Interactions for High-Performing Prompt Generation	44
5.1	Introduction	44
5.2	Background Literature	46
5.3	Methodology	46
5.3.1	Conversation Logs	47
5.3.2	Experimental Procedure	47
5.4	Results	47
5.4.1	High Performing Prompts for GPT-3.5-Turbo	48
5.4.2	High Performing Prompts for Mistral-V0.3-7B	49
5.4.3	High Performing Prompts for Llama-3.1-8B	51
5.4.4	Summary of Differences	53
5.5	Discussion	55
6	Conclusion and Future Work	58
6.1	Summary	58
6.2	Contributions	60
6.3	Future Work	61
A	Source Code	69
B	Agents	70
B.1	Prompt Design	70
B.2	HumanEval	71
B.3	GSM8K	72
B.4	SST-2	73
C	Task Prompts	74
C.1	Traditional Prompts	74

C.2 Claude-3-5-Sonnet Prompts	76
C.3 GPT-4o Prompts	90

List of Figures

3.1	Authoritarian framework architecture with three advisors.	18
3.2	Market-based framework architecture with three experts bidding on the prompt.	19
3.3	Hierarchical framework architecture with two teams consiting of three workers each.	20
3.4	Visualisation of prompt effectiveness when used with GPT-3.5-Turbo	30
4.1	Visualisation of prompt effectiveness when used with Mistral-V0.3	42
4.2	Visualisation of prompt effectiveness when used with Llama-3.1-8B	43

List of Tables

3.1	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for GSM8K	24
3.2	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet Approaches for SST-2	24
3.3	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet Approaches for HumanEval .	25
3.4	Latency results for GSM8K task using GPT-4o and Claude-3-5-Sonnet agents.	26
3.5	Latency results for SST-2 task using GPT-4o and Claude-3-5-Sonnet agents.	26
3.6	Latency results for HumanEval task using GPT-4o and Claude-3-5-Sonnet agents. . . .	27
4.1	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for GSM8K with Mistral-V0.3-7B	35
4.2	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for SST-2 with Mistral-V0.3-7B	35
4.3	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for HumanEval with Mistral-V0.3-7B	36
4.4	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet Approaches for GSM8K with Llama-3.1-8B	37
4.5	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for SST-2 using Llama-3.1-8B	37
4.6	Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for HumanEval with Llama-3.1-8B	38

Chapter 1

Introduction

This chapter provides an overview of the thesis by discussing the research motivation, objectives, experiments, contributions, and structure. It begins with a brief introduction to prompt engineering and its importance in improving LLM performance through optimised prompts. Next, the chapter outlines the study's objectives, describes the experiments conducted, and highlights the key contributions. The chapter concludes with an outline of the thesis structure.

This thesis explores the use of multi-agent systems to automate the optimisation of LLM prompts. The quality and effectiveness of prompts is essential to LLM performance, particularly when it comes to complex task where detailed guidance is required. Due to the enormity of the prompt search space the process of refining prompts can be a long and arduous task often yielding results that do not reflect the effort invested, hence the growing appeal of automating the process. This research on automated prompt optimisation first introduces three multi-agent architectures to optimise prompts and evaluates prompt performance with a large third-party model, it then determines the effectiveness of generating prompts for smaller more accessible models, and concludes with an exploration of conversational dynamics between the agents. A review paper will be written in the area.

This study was completed in collaboration with ZILO under the supervision of Sergio Alvares and Gamal Crichton, facilitating expert guidance and understanding, and validating the need for this study.

1.1 Research Motivation

The capabilities of LLMs have significantly advanced due to both the scaling of model parameters and the adoption of attention-based architectures [30, 23], revolutionising various aspects of society. Despite these advancements, consistently guiding LLMs to produce desired responses remains a considerable challenge. Fine-tuning LLMs is one approach to achieving this [76]; however, this method is often impractical or unattainable for most users. To address this, natural language prompts are used to guide LLMs, giving rise to the field of prompt engineering.

Prompt engineering requires crafting prompts that are clear, concise, and highly tailored to the task

at hand. However, this process is often complex and ambiguous, requiring significant experimentation. Given the challenges, there is a growing interest in automated prompt engineering, where algorithms can refine prompts more efficiently and effectively than humans. Approaches such as evolutionary algorithms [24, 21] and reinforcement learning [16] have been proposed, but this thesis focuses on using multi-agent conversational architectures, tackling domain adaptable prompting.

The primary goal of prompt engineering is to elicit desired responses from LLMs, with success measured by task-specific metrics. Although numerous benchmark datasets and LLM evaluation frameworks exist to aid in prompt evaluation, best practices for designing prompts are often generic and not universally applicable. Each LLM has unique sensitivities, responding differently to variations in prompt syntax and structure [13]. Therefore, prompt design requires careful experimentation and model-specific adjustments to achieve optimal performance.

The pursuit of artificial general intelligence (AGI) continues to drive rapid innovation in language models. As LLM capabilities expand, the complexity of tasks and use cases will increase, making prompt engineering increasingly critical. Across a range of domains, non-technical users will need to optimise and customise prompts to perform their tasks effectively. Ensuring error reduction and quality control will be crucial for producing reliable LLM outputs.

This research is motivated by the anticipated increase in inexperienced users, in a variety of domains, who will need to develop effective domain-specific prompting skills. Despite the growing importance of prompt engineering, there is currently no standardised approach to crafting prompts. To address this gap, this research proposes three unique, user-friendly, and effective multi-agent conversational frameworks for domains-specific automated prompt optimisation. The study seeks to evaluate the performance of different LLMs as agents within these frameworks, assess the effectiveness of engineered prompts in improving task performance for more accessible SLMs, and provide insights into the actions taken by agents to reinforce or uncover new best practices in prompt design.

Future research could extend this work by exploring the optimisation of prompts for complex real-world tasks and enhancing the current frameworks with advanced functionalities such as dynamic memory, precision edits, tool integration, mutable agents, agent-controlled team generation, and model-specific prompt tailoring.

In conclusion, as the AI landscape continues to evolve rapidly, prompt engineering will become increasingly important. This research offers a domain-adaptable solution to the challenges of prompt formulation by formalising three multi-agent conversational frameworks for autonomous prompt opti-

misation. Automating prompt engineering is a key factor in improving LLM usability across various domains, and this field is expected to grow and evolve alongside the LLMs it seeks to optimise.

1.2 Research Objectives

The primary objective of this research is to explore the capabilities of three novel multi-agent frameworks in autonomously optimising prompts, with the broader goal of contributing to the standardisation of prompt engineering practices. This objective is pursued through a series of experiments designed to evaluate the performance of these frameworks across diverse task domains and inference models, as well as to understand the impact of different LLMs when used as agents within these frameworks.

First, the research conducts a comparative analysis of the proposed frameworks against traditional prompt improvement techniques using common benchmark datasets. This analysis aims to quantify the effectiveness of the optimisation frameworks by examining their performance on established metrics. Additionally, the study investigates the performance of LLMs, namely GPT-4o and Claude-3-5-Sonnet, when they are employed as agents within these frameworks.

Next, the research extends the assessment of the frameworks to open-source SLMs, focusing on evaluating the effectiveness of the generated prompts when utilised by less powerful models. This aspect of the research seeks to determine how well the optimisation frameworks perform in scenarios involving weaker models, which are more accessible to a broader range of users.

Finally, the research includes a detailed study of the conversational dynamics between agents during the prompt optimisation process. This study aims to uncover how agents evaluate and refine prompts, identifying which aspects of prompt design are prioritised during optimisation. The goal is to determine whether these multi-agent interactions reveal new best practices in prompt engineering or reinforce existing ones.

1.3 Research Experiments

The research begins with a general analysis of the performance of the proposed frameworks by assessing optimised prompt outcomes on common benchmark datasets. It also evaluates the efficacy of two state-of-the-art LLMs when used as agents within these frameworks. Subsequently, the research examines the impact of framework generated prompts on open-source SLMs, investigating their ability to compete with proprietary third-party models. The study concludes with an analysis of the conversational dynamics

between the agents during the prompt optimisation process.

1. **Authoritarian, Market-Based and Hierarchical Optimisation Frameworks.** The first experiment formalises the multi-agent frameworks, highlighting key architectural features and testing them using GPT-4o and Claude-3-5-Sonnet as agents. Although novel in the context of automated prompt engineering, these frameworks are inspired by existing multi-agent architectures. The literature on automated prompt engineering reveals very few multi-agent approaches, and they neglect the domain adaptability of conversational agents. In this research, the frameworks employ agents that specialise in various aspects of prompt design and task domains. The prompts generated by these frameworks are tested with GPT-3.5-Turbo across a trio of diverse tasks using the GSM8K, SST-2 and HumanEval benchmarks. The performance of these prompts is compared against traditional prompt improvement techniques including zero-shot chain-of-thought and emotive prompting.
2. **Enhancing Open-Source SLMs to Rival Third-Party Models.** The second experiment examines the effectiveness of prompts generated by the frameworks when utilised by two open-source SLMs, namely Mistral-V0.3-7B and Llama-3.1-8B. The same trio of tasks from the first experiment is used, with a comparative analysis performed across different models and prompts. The LLM industry is highly competitive, and while the best-in-class models are not accessible to everyone, there has been little research on how automated prompt optimisation can enhance the performance of SLMs that require less computational power. This study aims to demonstrate how these small, accessible models can be upskilled using the prompts generated by the multi-agent frameworks to compete with proprietary LLMs that typically incur financial costs.
3. **Agent Interactions for High-Performing Prompt Generation.** The final experiment focuses on analysing the interactions between LLM agents during the prompt optimisation process. By examining conversation logs between agents in instances where the resultant prompt is highly effective, the study seeks to uncover the techniques and strategies employed by the agents. Existing research in prompt engineering often relies heavily on assumptions derived from human-human interactions, which may not fully apply to LLMs. This experiment aims to identify new and effective general, task-specific, and model-specific approaches to prompting implemented by LLM agents that may not be readily apparent to human prompt engineers.

1.4 Scientific Contributions

The research makes several contributions to science:

1. **Development of domain-adaptable prompt optimisation frameworks.** This study introduces three innovative frameworks for autonomously optimising prompts, designed with domain adaptability in mind. These frameworks leverage the capabilities of multiple conversational agents, each characterised by a distinct persona, to facilitate targeted prompt refinement. The efficacy of conversational agents in task completion is well-documented, with evidence indicating that the definition of agent personas can enhance performance. The adaptability of these personas further enables their application across diverse domains. However, the exploration of multi-agent systems for prompt optimisation remains under-explored in existing literature.
2. **Comprehensive evaluation of multi-agent architectures for prompt optimisation.** Following the formalisation of the aforementioned frameworks, this research conducts an extensive evaluation to determine the effectiveness of each framework, across both third-party LLMs and open-source SLMs, and identify the optimal models for powering the agents. The assessments are conducted across a diverse array of tasks, providing a broad perspective on performance. The study seeks to elucidate which multi-agent architectures are best suited to prompt optimisation, how different LLM agents effect the generated prompts, and if the frameworks can generate prompts capable of bridging the performance gap between state-of-the-art LLMs and accessible SLMs.
3. **Identification of best practices employed by LLM agents during prompt optimisation.** The thesis culminates in a detailed analysis of the conversational content and dynamics between agents during the prompt optimisation process. This analysis uncovers the strategies employed by the agents in engineering effective prompts, thereby revealing best practices that are emergent from the agents' interactions. Unlike current best practices, which are typically human-defined and inspired by human-human interactions, the practices observed in this study offer a novel perspective shaped by the agents' autonomous behaviors.

1.5 Thesis Structure

This thesis is organised as follows:

- **Chapter 2 - Background and Literature Review.** This chapter provides a comprehensive review of the literature on prompt engineering, emphasising its significance for LLMs and evaluating

current automated methodologies. The review identifies key challenges and opportunities within the field, highlighting motivations for this research.

- **Chapter 3 - Authoritarian, Market-Based, and Hierarchical Optimisation Frameworks.**

The first empirical study introduces three novel frameworks for automated prompt optimisation, utilising a multi-agent approach. Each framework is characterised by a distinct architecture, and their performances are systematically compared against each other and traditional prompt improvement techniques across a diverse set of datasets. The chapter also assesses the effectiveness of two state-of-the-art LLMs in serving as agents within these frameworks.

- **Chapter 4 - Enhancing Open-Source SLMs to Rival Third-Party Models.** This chapter presents the second study, which extends the findings of the first by focusing on the efficacy of prompts generated by the aforementioned frameworks when applied to small open-source models. The study examines the potential of these models to deliver performance comparable to that of proprietary models when using the framework optimised prompts, despite requiring significantly less computational resources.

- **Chapter 5 - Agent Interactions for High-Performing Prompt Generation.** The final study investigates the critical strategies employed by the optimisation frameworks to generate effective prompts across various tasks. Through detailed analysis of conversation logs, the chapter identifies key agent behaviors and actions that contribute to successful prompt generation.

- **Chapter 6 - Conclusions and Future Directions.** The thesis concludes with a synthesis of the research findings, highlighting the contributions of each study to the field of prompt engineering. It also discusses the limitations of the current work and offers suggestions for future research directions, aiming to advance the development of more robust and efficient prompt optimisation techniques.

Chapter 2

Background and Literature Review

This chapter presents background information on several key concepts central to this research. The chapter begins with the unprecedented rise of LLMs motivating research in prompt engineering, and then provides an overview of the prompting landscape and best practices in prompt engineering. It then reviews current approaches to automated prompt optimisation, including evolutionary algorithms, reinforcement learning, and gradient-based methods before extending the discussion to multi-agent systems, emphasising the use of LLM agents to solve complex problems.

2.1 Emergence of LLMs

The evolution of language models has undergone significant transformation since the 1990s, beginning with Statistical Language Models (SLMs) that used probabilistic methods to estimate sentence likelihoods based on preceding words, albeit constrained by storage limitations [54]. Neural Language Models (NLMs) introduced the use of neural networks, enabling the processing of longer sequences and the creation of word vectors [4, 43], with innovations like Word2Vec clustering semantically similar words [6]. These models set the stage for Pre-trained Language Models (PLMs), which, after extensive training on large unlabeled corpora, could be fine-tuned for specific tasks, advancing the field further [52].

Large Language Models (LLMs), such as GPT-3 [8], GPT-4 [46], PaLM [14], and Llama [64], represent the latest evolution, leveraging vast text corpora and billions of parameters. These models are pre-trained on extensive data and aligned with human values using techniques like reinforcement learning [76]. LLMs' adaptability is a result of increased model size, enhanced computational capabilities, and the availability of diverse internet-based data sources [38]. The advent of specialised hardware and the Transformer architecture [65], which allows for parallel processing and captures long-distance dependencies, has significantly refined LLM efficiency and effectiveness.

The versatility of LLMs is evident across domains such as finance, where they enhance modeling and risk management [40]; drug discovery, where they accelerate the identification and design of drug candidates [47, 56]; medicine, where they aid in diagnostics [63]; and law, where they streamline legal research and

analysis [7, 15]. These applications underscore the transformative potential of LLMs.

However, LLMs face challenges, including fairness, safety, and intellectual property concerns. Bias in training data can lead to unfair treatment of marginalized groups [3, 17, 57], and hallucinations in model outputs can undermine reliability [28]. Mitigating these risks through techniques like Reinforcement Learning from Human Feedback (RLHF) is essential, though costly [76]. Additionally, the use of extensive data raises questions about privacy and intellectual property, necessitating careful consideration for the ethical advancement of LLMs [31].

Guiding LLMs in an appropriate manner can prove a cost-effective approach to overcoming issues with reliability and bias [53, 29]. While the capabilities of LLMs are vast, their effectiveness is largely contingent upon the quality of the prompts they receive. This has led to the emergence of prompt engineering as a crucial practice, aimed at refining the inputs to guide these models toward producing reliable and contextually relevant outputs. The following section examines the methodologies and challenges of prompt engineering, which are essential for fully harnessing the potential of LLMs.

2.2 Prompt Engineering

Prompt engineering has emerged as a crucial technique for enhancing the capabilities of LLMs. It involves strategically designing task-specific instructions, referred to as prompts, to guide the model's output without altering its parameters. By leveraging well-crafted prompts, researchers and practitioners can significantly improve the performance of LLMs across various applications, ranging from natural language understanding to complex problem-solving tasks.

The principles of prompt engineering are rooted in the idea that the way a task is presented to an LLM can profoundly influence its performance. Effective prompts can clarify the task requirements, provide context, and reduce ambiguity, thereby enabling the model to generate more accurate and relevant responses. [9] suggests the key design principles of a prompt are conciseness and clarity, contextual relevance, task alignment, example demonstration, avoiding bias, and incremental prompting. It also touches on the use of programming logic for more complex tasks.

Prompt engineering capitalises on the inherent flexibility of LLMs, allowing them to adapt to a wide range of tasks with minimal adjustments to their underlying architecture. The foundational theories and research supporting the effectiveness of prompt engineering demonstrate its impact on a wealth of tasks [55].

The landscape of modern prompt engineering spans numerous techniques, each tailored to different

scenarios and objectives. Among these techniques are zero-shot, few-shot, chain-of-thought, and emotion-based prompting. Zero-shot prompting relies on the model's pre-existing knowledge and generalisation capabilities to understand and execute the task based solely on the prompt without demonstration. Few-shot prompting extends this by providing the model with a small number of examples within the prompt to illustrate the task [8]. Chain-of-Thought (CoT) prompting is a technique that involves structuring the prompt in a way that guides the model through a logical sequence of steps or reasoning processes [69]. Recent advancements of this include Tree-of-Thought (ToT) and Graph-Of-Thought (GoT) [73, 5]. Emotion-based prompting enhances LLM performance by combining the original prompt with emotional stimuli [34].

Both zero-shot and few-shot prompting are used to perform tasks without extensive training, whereas CoT, ToT and GoT are typically used when reasoning and logic is required [55]. This highlights the nuances of tailoring a prompt to the task at hand, which is not an easy feat and can require significant time and experimentation. Automating the process of engineering prompts has therefore emerged as an area of considerable interest.

2.3 Automated Prompt Optimisation

Prompt engineering has evolved significantly over the years, paving the way for LLMs to tackle a wider range of tasks with greater efficiency and adaptability. Part of this evolution has been automated prompt engineering, utilising LLMs, evolutionary algorithms, and reinforcement learning to autonomously optimise prompts, negating the lengthy and experimental nature of manual prompt engineering.

LLMs themselves have been shown to be effective tools in optimising prompts. Automatic Prompt Optimisation (APO) proposes a non-parametric approach inspired by numerical gradient descent [51]. Natural language "gradients" criticise the current prompt, and the gradients are then "propagated" into the prompt by editing the prompt in the opposite semantic direction of the gradient. The process is guided by a bandit selection and beam search procedure to improve algorithmic efficiency. The approach is task-agnostic; however, cross-task generalisation and robustness are not deeply explored in the paper. It also relies heavily on task-specific feedback signals, which may be sparse or noisy, resulting in a struggle to converge to effective prompts.

Automatic Prompt Engineer (APE) takes a different approach, performing optimisation by searching a pool of candidate instructions proposed by an LLM in order to maximise a chosen score function, with evaluation performed using zero-shot performance [75]. The authors conduct extensive experiments

across a variety of NLP tasks and settings demonstrating improvements over manual prompt engineering; however, there is a risk that LLMs might generate degenerate or non-sensical prompts.

The integration of LLMs with evolutionary algorithms (EAs) has also been recently explored as an effective approach to optimise prompts. These techniques utilise LLMs to iteratively refine prompts by simulating evolutionary processes, with concepts of mutations and survival of the fittest, leading to progressively better performance. Most notably, Microsoft proposes EvoPrompt for discrete prompt optimisation [24]. The process begins with a population of prompts and iteratively generates new prompts with LLMs based on evolutionary operators, improving the population based on a development set. It leverages EAs strength in exploring large, complex search spaces, but can be sensitive to the initialisation of the population and the specific hyperparameters used, leading to poor convergence. It also relies on having task-specific datasets which are not always readily available.

PromptBreeder is another intriguing use case of EAs, which employs self-referential self-improvement with LLMs for prompt evolution [21]. The self-referential self-improvement aspect behind PromptBreeder is that it does not just improve task-prompts, but also improves the mutation prompts that improve these task-prompts. The process is general purpose and provides robust exploration of the prompt space, but, as with EvoPrompt, poor initialisation or hyperparameter choices may lead to suboptimal results.

Reinforcement learning (RL) presents another promising avenue for prompt optimisation. RLPrompt formulates a parameter-efficient policy network that generates an optimised discrete prompt after reward training, and claims to be flexible in applying to different types of models for both classification and generation tasks [16]. The study shows that optimised prompts, even if they look like gibberish, can transfer between different LLMs while retaining effectiveness. However, the success of RLPrompt is heavily based on the design of effective reward functions, which may require domain expertise.

Prompt-OIRL harnesses offline inverse reinforcement learning to draw insights from offline prompt demonstration data, with the query-dependent prompt optimisation objective achieved by first learning an offline reward model [60]. The model can evaluate any query-prompt pairs without accessing LLMs and shows versatility across different models and tasks. However, the success of the offline IRL approach heavily depends on the quality and diversity of the pre-collected data, and designing effective reward functions that generalise well across different queries and tasks.

The use of LLM agents has additionally been investigated to automate prompt optimisation, exploiting techniques including behavioural alignment and self-supervision, and incorporating domain knowledge. PromptAgent views prompt optimisation as a strategic planning problem and employs a principled

planning algorithm while using the self-reflection abilities of LLMs [67]. The approach demonstrates improved accuracy across various tasks, particularly in the biomedical and complex reasoning domains, by effectively leveraging domain knowledge. However, the method requires highly comprehensive training data and may require a sophisticated understanding and resources to implement effectively, which could be a barrier for some users or organisations lacking technical expertise.

Mixture-Of-Experts in prompt optimisation aims to overcome the restricted output space of the search problem in other optimisation methods [66]. It partitions the problem space into multiple homogeneous regions, each governed by a specialised expert agent, and the prompt optimisation problem becomes an expert assignment problem. By employing a mixture of prompts, each expert can focus on a specific region of the problem space allowing for tailored instructions that enhance performance on diverse tasks. However, the effectiveness of the approach is highly dependent on the quality of the training demos and evaluating the performance of multiple experts and their interactions can be more challenging than assessing a single prompt.

Each of the existing approaches to optimise prompts brings unique advantages; however, common downsides include sensitivity and complexity of initialisation, and the need for quality and comprehensive training data. It's also apparent mechanisms to prevent nonsensical prompts and incorporate domain knowledge are on the main neglected. The application of multi-agent systems presents an intriguing opportunity to address some of the limitations.

2.4 Multi-Agent Systems

Single- and multi-agent approaches have scarcely been researched for automated prompt optimisation. However, as the field of AI continues to evolve and LLMs are expected to perform tasks of increasing complexity in other domains, researchers are frequently relying on the strengths of multi-agent systems (MAS) where multiple LLM entities collaborate towards a common goal.

MAS represent a sophisticated approach to enhancing the capabilities of LLMs by leveraging the collaboration and interaction among multiple LLM agents. They can be considered to operate in a black box since outputs are generated based on complex internal computations that are not directly observable. Agents communicate, exchange information, and perform tasks within this black box environment, potentially augmented by tools that provide additional functionality.

The difference between a single agent systems (SAS) and MAS is that SAS are powered by one language model which performs all tasks on their own. Notable effective single agent architectures include ReAct,

RAISE, and Reflexion. In a ReAct architecture, an agent first writes a thought about the given task then performs an action based on that thought [72]. RAISE is an enhancement of the ReAct framework incorporating a dual-component memory system mirroring human short-term and long-term memory [36]. Reflexion utilises the agent’s ability to self-reflect on its actions and outputs [58]. Agents use feedback signals to reflect on tasks and store their reflections in memory to improve future decisions. Single-agent methodologies provide the groundwork upon which multi-agent systems are built.

A robust framework for multi-agent systems includes several key components, as detailed in [61]. The black box environment can be represented as a graph, $G(V, E)$, where nodes represent agents and edges represent possible interactions. Agents are represented as a tuple, $A_i(L_i, R_i, S_i)$, encapsulating the LLM, role, and current state of the agent. Tools have a similar representation, $T_j = (F_j, C_j, U_j)$, encapsulating the functionalities, configuration and usage constraint. Inter-agent feedback and self-feedback mechanisms are hugely important to MAS performance, facilitating discussions and self-reflection to foster continuous improvement [22][41]. It is possible for a system to support the dynamic addition of agents to aid collaboration and workload management, enabling scalability and adaptability, and halting mechanisms and supervision ensure that agents know when to stop their activities, ensuring efficient task completion [61].

The coordination of agents in MAS should be determined by the goal and domain of task [49]. Vertical architectures are those in which one agent acts as a leader and has other agents report directly to them, and are suited to tasks that require planning and execution in dynamic environments [2]. Conversely, in horizontal architectures all the agents are treated as equals existing in a group discussion about the task, and are generally effective when collaboration and feedback are key to the overall success of the task [12]. [32] highlights how organisational structure can be implemented in MAS, including hierarchical and bidding style architectures.

Utilising LLMs in MAS is proving effective in enhancing performance across a range of tasks. The Dynamic LLM-Agent Network (DyLAN) framework aims to improve performance on complex tasks, including reasoning and code generation, by creating a dynamic agent structure [39]. The framework assesses agent contributions in previous rounds and selects the best agent for subsequent rounds. Additionally, AgentVerse uses group planning to enhance reasoning and problem-solving capabilities of LLM agents [12], and MetaGPT [26] addresses the issue of unproductive chat among agents by enforcing the generation of structured outputs instead of sharing unstructured chat messages.

With the growing popularity of multi-agent LLM frameworks, there are multiple tools to effectively

implement them including LangChain¹, LangGraph² and CrewAI³. Each provides building blocks to inter-connect LLM agents, streamlining the process of creating complex LLM-powered MAS. Notably, LangGraph uses a graphical representation similar to the above for agent coordination, offering a clear and scalable method to manage multi-agent interactions.

The growing usability and strengths of MAS suggest applicability to automated prompt engineering and potential to address some downfalls of existing approaches. Introducing agents with domain-specific personas encourages domain-specific prompts, increasing the efficacy of the prompt search. The concept of local and global memory further helps to target and speed up the search. Multiple agents contributing to the optimisation process also allows for continuous error checking helping to prevent non-sensical prompts or hallucinations.

¹<https://github.com/langchain-ai/langchain>

²<https://github.com/langchain-ai/langgraph>

³<https://github.com/crewAIInc/crewAI>

Chapter 3

Authoritarian, Market-Based and Hierarchical Optimisation Frameworks

This study introduces three innovative multi-agent frameworks for automated prompt optimisation, with a focus on the strategic implementation of agent roles and core principles for domain adaptability. The proposed frameworks encompass three distinct architectural paradigms: Authoritarian, Market-Based, and Hierarchical. The research first formalises these frameworks, detailing their structural designs and the interactions between agents. Subsequently, the study empirically evaluates the efficacy of these frameworks by employing them with GPT-4o and Claude-3-5-Sonnet to enhance baseline prompts in the domains of code generation, mathematical reasoning, and sentiment analysis. The optimised prompts are systematically evaluated against each other, as well as against a series of conventional prompt improvement techniques. The findings reveal notable variations in the effectiveness of prompts generated by each framework and by GPT-4o and Claude-3-5-Sonnet when used by GPT-3.5-Turbo for inference. Notably, the Hierarchical framework generates much shorter prompts than the Authoritarian and Market-Based frameworks that seem to be favoured by GPT-3.5-Turbo, and prompts generated using Claude-3-5-Sonnet for code generation exhibit marked improvements over the Baseline, but results were not statistically significant. The study demonstrates the capabilities of the multi-agent frameworks to generate domain-specific, error-free prompts consistently, and offers intriguing insight into the approaches taken by GPT-4o and Claude-3-5-Sonnet.

3.1 Introduction

This initial study investigates the effectiveness of multi-agent conversational architectures in autonomous prompt optimisation. Specifically, it formalises three distinct multi-agent architectures: Authoritarian, Market-based, and Hierarchical, and tests them using two third-party models. The primary objective is to assess and compare the performance of the architectures in optimising prompts for different domains, and determine which model operates best as the conversational agents. Three tasks, namely code

generation, mathematical reasoning, and sentiment analysis, are used for evaluation. Understanding the differences in these architectures and the models that power them is crucial for optimising their application and addressing existing issues in autonomous prompt generation.

Existing studies have explored various evolutionary algorithms and reinforcement learning approaches to automate prompt engineering, but few draw on the strengths of multi-agent systems. There is a lack of comprehensive experimentation and analysis into multi-agent architectures despite their proven ability to coordinate complex tasks. Many previous works neglect the importance of domain-adaptability and error checking in their frameworks, implementing a task-agnostic process without the ability to tailor it; oversights that could be tackled using multiple agents. This study aims to address these issues by providing a unified context for evaluating the performance of three domain-adaptable multi-agent frameworks, each with distinct architectures.

Simplicity and transparency are favoured in the methodology which involves formalising the multi-agent architectures, producing prompts using two state-of-the-art LLMs, and evaluating their performance on three diverse tasks with a third state-of-the-art LLM. The performance of the architectures is expected to outperform select traditional prompt improvement methods, namely a zero-shot baseline, zero-shot-chain-of-thought, and emotive prompting. A simple exact match accuracy metric is used to assess the effectiveness on each task. By systematically comparing the results, the aim is to identify the strengths and weaknesses of each architecture and model, and determine the potential of the frameworks.

Motivations for this study include the need for a fast method to find optimised prompts amidst the vast prompt search space, similar to existing studies. However, this study looks to act on the above mentioned oversights and address the lack of domain-adaptable approaches and vulnerability to non-sensical prompt generation. Previous research has capitalised on the strengths of multi-agent LLM systems in coordinating and completing tasks end-to-end, but there is a lack of studies exposing them to prompt optimisation, despite their apparent compatibility.

This study aims to introduce three novel domain-adaptable multi-agent frameworks to autonomously optimise prompts, and establish strengths and weaknesses of each. The performance of Authoritarian, Market-Based, and Hierarchical architectures are assessed on benchmark datasets and compared against traditional prompt improvement techniques. The rest of the study is organised as follows: section 2 provides a focused literature review; section 3 details the methodology; section 4 presents the results; and section 5 concludes with a discussion of findings.

3.2 Background Literature

The literature review begins by expanding on the review of multi-agent systems (MAS) in Chapter 2, focusing on existing applications. It then presents a fine-grained discussion of LLM-driven MAS and the frameworks that exist to implement them. To conclude it highlights current multi-agent setups to optimise prompts.

MAS have gained significant attention in recent years due to their ability to solve complex problems through the cooperation of multiple autonomous agents [18]. These systems are characterised by their decentralized nature and the capability for agents to interact, collaborate, and even compete. Recent studies underscore the effectiveness of multi-agent algorithms in cooperative settings. These algorithms have been shown to achieve strong performance in various applications, including within robotic teams [59], traffic control [62], and resource management [70], indicating their potential for broader implementation in real-world scenarios. Agents in MAS can be equipped with behaviors designed in advance, however, often agents are tasked with discovering a solution on their own. Hence, reinforcement learning is often coupled with MAS such that agents learn as the performance of a system improves by interacting with its dynamic environment [10]. [25] highlights how deep reinforcement learning can facilitate cooperative control among agents, thereby enhancing their collective decision-making capabilities.

The integration of LLMs into multi-agent systems introduces a novel paradigm for collaboration among agents. The MetaGPT framework exemplifies this, as it incorporates human workflows into LLM-based multi-agent collaboration [26]. This framework allows for a meta-programming approach that enhances the efficiency of agent interactions, thereby addressing the complex demands of contemporary applications. The ability of agents to engage in dialogue and negotiate solutions represents a significant leap forward in leveraging LLM capabilities for multi-agent tasks. Additionally, the ChatEval framework, which employs a multi-agent debate model, showcases the potential of using MAS for evaluating the quality of generated responses from LLMs [11]. This debate framework allows agents to autonomously discuss and critique outputs, thereby fostering a more robust evaluative process. Such frameworks not only enhance the quality of LLM outputs but also encourage divergent thinking [35]. Other LLM-MAS have shown remarkable performance in tasks including multi-robot-systems and orchestrating simulations [42, 48, 27, 71].

Automated prompt optimisation is emerging as a promising area of research to enhance the capabilities of LLMs. MAS have been investigated but remain relatively overlooked in the context of automated prompt optimisation. [66] investigates multi-agent collaboration, introducing a novel approach that leverages

the Mixture-of-Experts (MoE) paradigm to enhance prompt design. This work addresses the limitations of traditional prompt engineering by providing a more tailored and effective strategy for optimising prompts across complex tasks. [50] also implements a MoE architecture, introducing a novel method for prompt optimisation that utilises structured, principle-based edits to enhance prompt effectiveness. It allows for the creation of specialised prompts tailored to distinct semantic regions of the training data.

Despite promising findings in the application of MAS for automated prompt optimisation, several knowledge gaps remain. While the existing literature demonstrates the enhancements in performance via prompt generation through MoE collaboration, there is limited research into other multi-agent architectures and the impact they have on prompt generation. Investigating how different configurations of agents perform in diverse contexts would provide valuable insights into fine-tuning multi-agent approaches for prompts optimisation. It's also clear that LLM agents, or experts, help to target the optimisation process and behavioural alignment should be exploited.

3.3 Methodology

To systematically evaluate the performance of the different multi-agent architectures, a simple but comprehensive methodology is designed encompassing several key components. This section outlines the formalisation of the multi-agent architectures, the selection of benchmark datasets, the evaluation metrics used to assess performance, and the detailed steps of the experimental procedure.

3.3.1 Formalisation of Multi-Agent Architectures

This study evaluates three distinct multi-agent architectures — Authoritarian, Market-Based, and Hierarchical — chosen for their diversity in agent coordination. Each architecture aims to leverage the strengths of using multiple agents to effectively approach prompt optimisation, but through contrasting methodologies.

LangGraph offers fine-grained control over both the flow and state of applications, and is therefore used to implement the frameworks. It allows for the definition of chains of computation, as well as the addition of cycles, enabling more complex, agent-like behaviors. The architectural designs are inspired by LangChain and LangGraph multi-agent demonstrations¹.

All frameworks share several key features: they define agent roles and core principles, employ continuous intrinsic evaluation, dynamically control the process, and use an SQLite database for efficient memory

¹<https://github.com/langchain-ai>

storage and retrieval. Only the message history and next advisor are maintained in the graph state in each framework. Notably, agent roles introduce the ability to tailor the process to different domains, and agent interactions coupled with global memory helps correct non-sensical actions if they take place.

Each framework employs a set of agents specialised in various aspects of prompt design, including style, structure, clarity, relevance, and task-alignment [9]. Additionally, domain-specific agents are used depending on the task to encourage targeted exploration of the prompt space and the incorporation of domain-specific knowledge. For more details, refer to Appendix B.

In the Authoritarian framework (Figure 3.1), a single authoritative agent dictates prompt changes with input from advisory agents. The authoritative agent reviews the prompt at each iteration, selects an advisory agent for feedback, and acts on the feedback received. At each iteration, the list of possible advisors for selection by the authoritative agent consists only of advisors that disapprove of the prompt to encourage focus on the weakest aspects of the prompt. To mitigate positional bias during agent selection, the list order is also randomised. The optimisation process is considered complete when the advisory agents unanimously approve the prompt, with a maximum of eight iterations to prevent excessive runs.

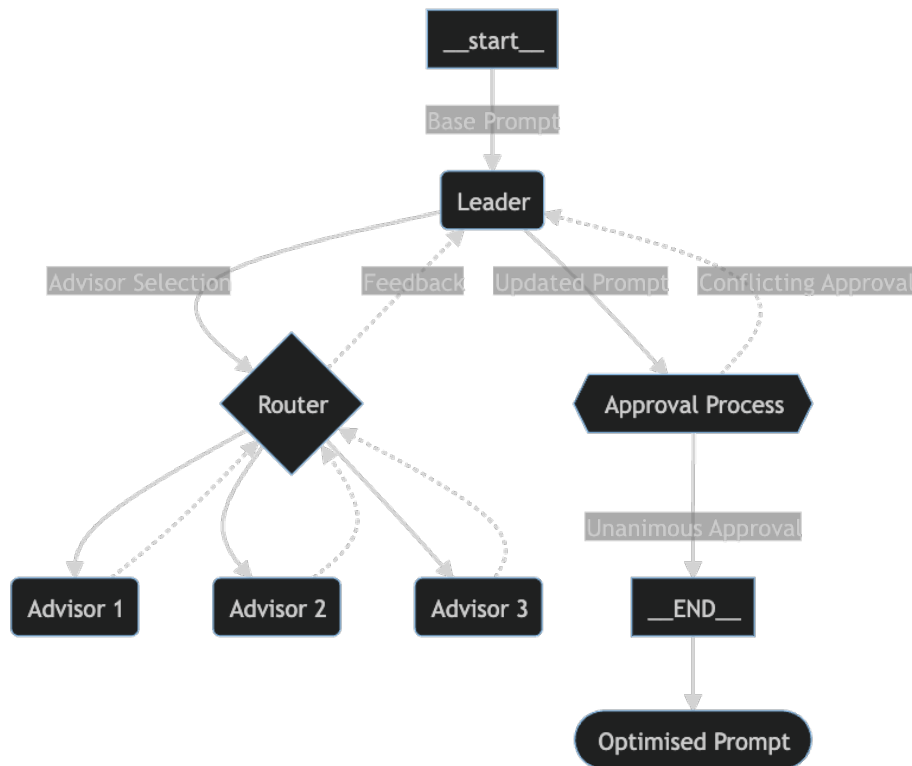


Figure 3.1: Authoritarian framework architecture with three advisors.

The Market-based architecture (Figure 3.2) operates a competitive environment where multiple agents bid on improving the prompt. The higher the bid, the more impact the agent thinks it can have on the prompts effectiveness. The highest bidder then updates the prompt based on their role and core principles, using a self-reflection graph consisting of a generation and reflection node to ensure updates are role-specific and adhere to set guidelines. The optimisation concludes when all bids fall below a threshold value of 2, indicating all agents believe the prompt requires no further improvement.

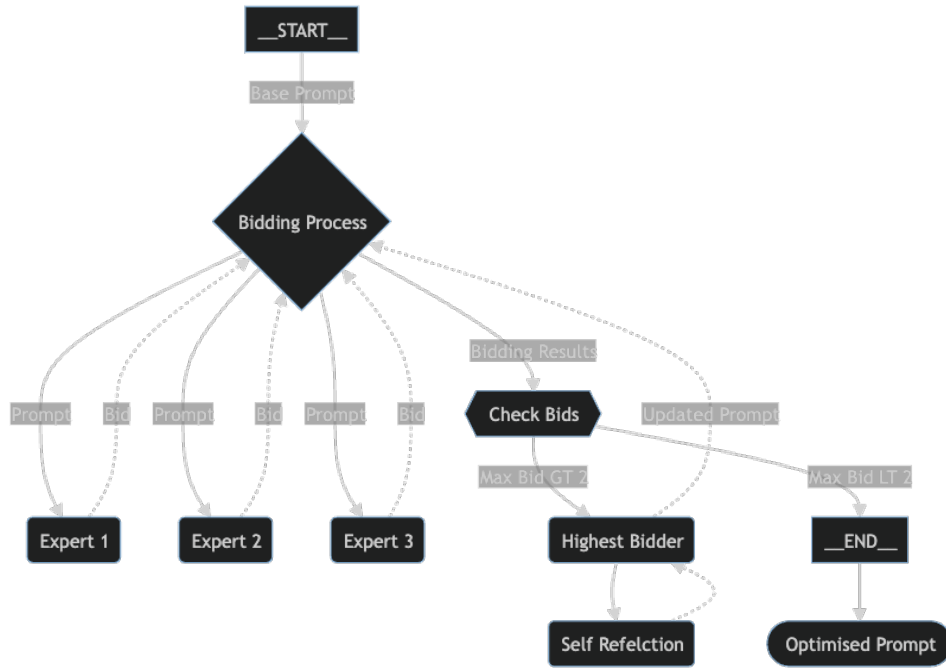


Figure 3.2: Market-based framework architecture with three experts bidding on the prompt.

In the Hierarchical framework (Figure 3.3) agents are organised in a tiered structure. A top-level agent oversees two teams — a domain team and a prompt design team — each led by a team leader. The team leaders coordinate prompt updates within their teams and request feedback before presenting the prompt to the top-level agent for a final review. The team leader can perform a maximum of three feedback iterations before feeding the updated prompt back to the top level agent. As in the Authoritarian framework, the top-level agent can only select a team leader if they disapprove of the prompt, and to counter positional bias, the order of team leaders is randomised. The process terminates when both team leaders approve the prompt, with a maximum of four iterations to manage complexity.

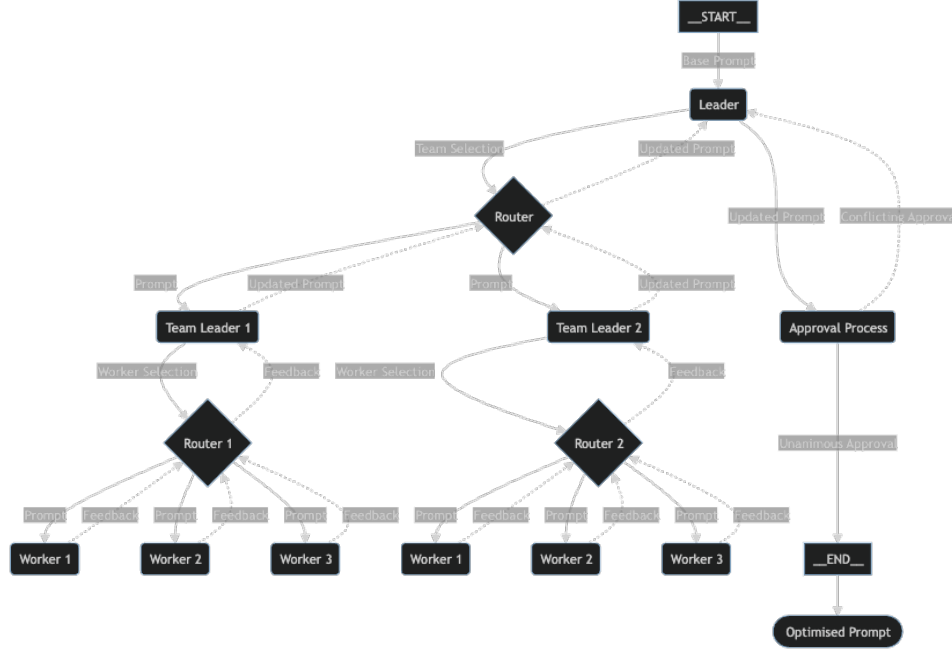


Figure 3.3: Hierarchical framework architecture with two teams consisting of three workers each.

3.3.2 Benchmark Datasets and Evaluation

Framework evaluation employs three well-established benchmark datasets, each targeting a distinct type of task: code generation, mathematical reasoning, and sentiment analysis. Performance on each dataset is assessed using exact match accuracy scores. To facilitate this, LLM outputs are pre-processed, and scores are contingent upon the accuracy of outputs according to predefined regular expressions.

For code generation tasks, the HumanEval dataset is utilised. This dataset evaluates the ability of the model to complete functions given a function signature and docstring. Each function must pass a series of assertion tests to be considered correct. Accuracy is calculated as the proportion of code snippets that pass all assertion tests. The evaluation framework is provided by the HumanEval GitHub repository²,

²<https://github.com/openai/human-eval>

which defines the generation and testing processes. To enhance execution efficiency and safety, minor adjustments were made to run code in parallel within a Docker container.

The GSM8K dataset is used to evaluate mathematical reasoning. Here, the LLM is required to produce the correct numerical answer in the specified format. Accuracy is determined by the proportion of correct numerical answers produced by the model. This dataset provides a robust measure of the model’s ability to handle arithmetic word problems.

Sentiment analysis is assessed using the SST-2 dataset, which involves classifying text as either positive or negative. Responses are mapped to binary values (1 for positive, 0 for negative). Accuracy is calculated as the proportion of correct classifications. Evaluation for the GSM8K and SST-2 datasets is conducted using Microsoft’s PromptBench³ framework.

The selection of these datasets ensures a broad evaluation of the multi-agent frameworks’ capabilities across diverse domains. By applying these datasets and evaluation methods, the study aims to deliver a comprehensive initial assessment of the performance of the multi-agent prompt optimisation frameworks.

3.3.3 Experimental Procedure

The primary objective of the experiment is to compare the effectiveness of the three multi-agent frameworks in optimising prompts and demonstrate the ability to adapt the frameworks to the domains of code generation, mathematical reasoning and sentiment analysis. The study tests the performance of the frameworks using GPT-4o then Claude-3.5-Sonnet as agents with the goal of determining which model best suits the task. GPT-3.5-Turbo is used for inference since it is commonly used in the literature and is relatively cheap for a third-party model.

Initially, each framework is set up with a Baseline prompt derived from the dataset content: function signature and docstring for HumanEval, math word problem for GSM8K, and sentiment text for SST-2. Alongside the Baseline prompt, a list of success criteria is provided to guide the agents, specifying the expected outcomes and emphasising important task aspects.

Each multi-agent framework iterates on the Baseline prompt provided. Temperature controls the randomness of LLM outputs, and a lower temperature results in more deterministic responses. Preliminary tests indicated lower temperatures resulted in more reliable outputted prompts and therefore the temperature of all agents is set to 0.0. 0.0 is also the temperature used during inference. Even with a temperature of 0.0 LLM generation exhibits randomness so the process is repeated three times for each

³<https://github.com/microsoft/promptbench>

framework, generating three distinct prompts representing a range of outputs that can be expected. The generated prompts are then evaluated against a random sample of 30 test cases from each benchmark dataset. This sample size provides a sufficiently large set of test cases to observe the impact of prompts while managing financial constraints during inference.

Mean, median, and maximum scores along with standard deviations are recorded for each prompt to provide a comprehensive assessment of prompt performance. The Wilcoxon Signed-Rank Test is a non-parametric test that compares two paired groups and is used for significance testing. It is used here because the sample sizes are very small, it does not assume that the data is normally distributed, and it is suitable for comparing the differences between two related samples. By analysing these metrics, the study aims to identify the most effective framework and agent model for automated prompt optimisation across different domains.

3.4 Results

This section details the results of the optimisation processes, first from a qualitative perspective commenting on how each framework paired with GPT-4o and Claude-3-5-Sonnet agents configured prompts. It then details the quantitative results of using the prompts for inference with GPT-3.5-Turbo. Mean, median and maximum scores are presented for each prompting technique, along with the standard deviation between results. While maximum scores can be biased by exceptional cases, they provide valuable insight into the effectiveness of the frameworks. Since once an optimal prompt is identified it does not typically need to be altered, the maximum score is a valuable measure for real-world application. The section concludes providing information regarding the latency of the frameworks, further informing on operational efficiency.

3.4.1 Qualitative Analysis of Prompts

The prompts generated by GPT-4o and Claude-3-5-Sonnet (see Appendix C) exhibit notable similarities as well as distinct differences across the multi-agent frameworks. As hoped by using multiple agents and exploiting inter-agent feedback and error checking, all prompts are interpretable, make logical sense, and are compatible with the evaluation procedure with placeholders present in the Baseline remaining.

Under the Authoritarian framework, the prompts produced by GPT-4o are generally succinct, typically consisting of one or two lines with a straightforward task instruction followed by formatting details. In contrast, Claude-3-5-Sonnet’s prompts are significantly longer. They feature more elaborate structures,

often including bullet points or numbered instructions to clearly delineate sections. This approach demonstrates a preference for detailed and organised prompt layouts.

In the Market-Based framework, the length of prompts is more comparable between GPT-4o and Claude-3-5-Sonnet. However, the structural approach differs. GPT-4o’s prompts in this framework exhibit increased organisation compared to the Authoritarian setting, utilising more structured elements. Claude-3-5-Sonnet continues its trend of employing numbered instructions but also introduces additional creative elements such as ASCII visualisations, mistake spotlights, and self-reflection segments. GPT-4o’s prompts, while still incorporating numbered instructions, place a greater emphasis on example demonstrations.

The Hierarchical framework often results in noticeably shorter prompts for both GPT-4o and Claude-3-5-Sonnet. These prompts typically include a basic task description, formatting instructions, and brief guidance such as “show your workings” or “break down the problem,” or a concise example. The brevity and directness of prompts in this framework suggest a focus on clarity and simplicity.

Claude-3-5-Sonnet consistently adds more detail and creative flair to its prompts across all frameworks. Its prompts are often more structured and readable for human users, leveraging various formatting techniques to enhance clarity and organisation. In contrast, GPT-4o tends to produce more straightforward prompts with a focus on task-specific instructions and example demonstrations, reflecting a more minimalistic and direct approach. Claude-3-5-Sonnet’s propensity for detailed and creatively structured prompts may contribute to enhanced readability and context for complex tasks, while GPT-4o’s concise and structured prompts offer efficiency and directness.

These observations highlight the contrasting styles of prompt generation between GPT-4o and Claude-3-5-Sonnet, as well as between different architectures.

3.4.2 Quantitative Analysis of Prompts

The tables indicate the mean, median and max scores for each prompt tested, as well as the standard deviation between results. GPT-3.5-Turbo is the model used for inference and demonstrates exceptional performance with the Baseline and traditional prompt improvement techniques (ZS-CoT, Emotive). However, the automatically optimised prompts present the potential of the three frameworks, and establish how each framework and agent (GPT-4o or Claude-3-5-Sonnet) may suit different tasks.

For the GSM8K task (Table 3.1), while GPT-4o agent frameworks do not consistently surpass the traditional baseline’s peak performance of 90%, they show substantial improvement over ZS-CoT and

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	85.6%	86.7%	90.0%	4.1%
	ZS-CoT	63.3%	63.3%	66.7%	2.7%
	Emotive	74.4%	76.7%	76.7%	3.2%
GPT-4o	Authoritarian	68.9%	83.3%	86.7%	22.8%
	Market	73.3%	80.0%	80.0%	9.4%
	Hierarchical	80.0%	76.7%	90.0%	7.2%
Claude-3-5-Sonnet	Authoritarian	73.3%	76.7%	76.7%	4.7%
	Market	51.1%	73.3%	80.0%	36.2%
	Hierarchical	45.6%	50.0%	76.7%	27.4%

Table 3.1: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for GSM8K

Emotive. Hierarchical optimisation even matches the Baseline peak score with its highest performing prompt. However, when Claude-3-5-Sonnet agents are implemented, the performance complexity increases, especially under the Market and Hierarchical frameworks, which display substantial variance. Claude-3-5-Sonnet’s Authoritarian framework is competitive with a mean of 73.3%, in contrast to 51.1% and 45.6% for Market and Hierarchical frameworks respectively.

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	96.7%	96.7%	96.7%	0.0%
	ZS-CoT	62.2%	63.3%	63.3%	1.6%
	Emotive	96.7%	96.7%	96.7%	0.0%
GPT-4o	Authoritarian	93.3%	93.3%	96.7%	2.7%
	Market	93.3%	93.3%	96.7%	2.7%
	Hierarchical	94.4%	93.3%	96.7%	1.6%
Claude-3-5-Sonnet	Authoritarian	96.7%	96.7%	96.7%	0.0%
	Market	81.1%	80.0%	86.7%	4.2%
	Hierarchical	93.3%	93.3%	96.7%	2.7%

Table 3.2: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet Approaches for SST-2

In sentiment analysis using SST-2 (Table 3.2), all frameworks display high effectiveness. Both the traditional Baseline and Emotive prompts consistently achieve 96.7%. Prompts produced by GPT-4o agents within the Hierarchical framework closely follow with a mean of 94.4%, while those from the Authoritarian and Market frameworks scored 93.3%. Peak performance across all frameworks matches that of the Baseline and Emotive approaches. Claude-3-5-Sonnet agents within the Authoritarian framework produce prompts matching the Baseline score with a mean of 96.7%. Prompts from the Hierarchical framework also achieved an impressive 93.3%, but those from the Market framework performed notably worse.

When evaluating the seemingly more challenging HumanEval task (Table 3.3), prompts produced by GPT-4o agents within the Authoritarian framework improve upon the mean Baseline score (81.1%

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	80.0%	80.0%	80.0%	0.0%
	ZS-CoT	86.7%	86.7%	86.7%	0.0%
	Emotive	76.7%	76.7%	76.7%	0.0%
GPT-4o	Authoritarian	81.1%	80.0%	83.3%	1.6%
	Market	74.5%	76.7%	76.7%	3.2%
	Hierarchical	74.4%	76.7%	83.3%	8.3%
Claude-3-5-Sonnet	Authoritarian	72.2%	83.3%	86.7%	18.1%
	Market	77.8%	76.7%	80.0%	1.6%
	Hierarchical	82.2%	83.3%	83.3%	1.6%

Table 3.3: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet Approaches for HumanEval

vs. 80.0%). Once again, the Claude-3-5-Sonnet results paint a complex picture. Prompts from the Authoritarian framework exhibit highly competitive performance which, discounting an outlier, presents a compelling case for use over the traditional improvement techniques. The prompts achieve a median score of 83.3% and maximum 86.7%, which matches that of the top performing ZS-CoT prompt. Prompts resulting from the Hierarchical framework show more stability but only surpass the Baseline and Emotive performance, not ZS-CoT, achieving a mean score of 81.8%.

The Wilcoxon Signed-Rank Test suggests no significant differences between the mean results of each approach; however, the sample size of three makes it very difficult to comment on the significance of the results, and sensible analysis of the raw statistical values is possibly more informative.

Overall, the results highlight the high performing nature of GPT-3.5-Turbo on the set of tasks with very basic prompts. Such high scores prove difficult to surpass; however, many autonomously engineered prompts for sentiment analysis match the remarkable Baseline of 96.7%, and a good proportion of prompts for code generation improve on the Baseline of 80% but fail to exceed the ZS-CoT approach. The results offer encouragement for the capabilities of the frameworks.

3.4.3 Latency of Frameworks

The speed at which each framework optimises prompts is important for their practical application, and varies depending on the task and whether GPT-4o or Claude-3-5-Sonnet is used. The results are summarised in Table 3.4, Table 3.5, and Table 3.6. The results provide insights into the performance efficiency of the optimisation frameworks across the tasks, and make it clear how much faster they explore the prompt space than a manual approach.

For the GSM8K task, using GPT-4o agents results in notably lower latency across all frameworks compared to using Claude-3-5-Sonnet agents. The fastest approach when using GPT-4o is the Hierarchical

Agent	Framework	Mean	Median	Min	Std. Dev.
GPT-4o	Authoritarian	83.0s	38.3s	33.0s	67.0s
	Market	153.0s	147.1s	117.6s	31.6s
	Hierarchical	43.2s	40.3s	24.2s	16.8s
Claude-3-5-Sonnet	Authoritarian	251.7s	203.0s	197.3s	72.9s
	Market	185.0s	178.7s	175.3s	11.4s
	Hierarchical	233.9s	218.3s	209.7s	28.4s

Table 3.4: Latency results for GSM8K task using GPT-4o and Claude-3-5-Sonnet agents.

framework, displaying a mean latency of 43.2 seconds and a minimum of 24.2 seconds. On the contrary, the slowest approach using GPT-4o is the Market framework, with a mean latency of 153.0 seconds. With Claude-3-5-Sonnet, the Authoritarian framework exhibits the highest latency with a mean of 251.7 seconds, while the Market framework is the fastest with a mean latency of 185.0 seconds. Using GPT-4o within the Hierarchical framework is consistently markedly faster than any using Claude-3-5-Sonnet within any framework.

Agent	Framework	Mean	Median	Min	Std. Dev.
GPT-4o	Authoritarian	66.7s	77.9s	28.9s	27.4s
	Market	144.9s	151.4s	128.7s	11.6s
	Hierarchical	89.9s	43.1s	41.4s	67.4s
Claude-3-5-Sonnet	Authoritarian	181.8s	182.0s	176.3s	4.5s
	Market	186.0s	181.4s	175.4s	11.1s
	Hierarchical	263.5s	248.3s	201.6s	57.8s

Table 3.5: Latency results for SST-2 task using GPT-4o and Claude-3-5-Sonnet agents.

In the SST-2 task, using GPT-4o again results in lower latency across most frameworks compared to using Claude-3-5-Sonnet. The fastest approach using GPT-4o is within the Authoritarian framework, with a mean latency of 66.7 seconds and a minimum of 28.9 seconds. The slowest approach using GPT-4o is again the Market framework, with a mean latency of 144.9 seconds. For Claude-3-5-Sonnet, the Authoritarian framework has the lowest mean latency at 181.8 seconds, but is still noticeably slower than the fastest GPT-4o agent framework. This is in contrast to GSM8K where the Authoritarian framework is slowest. The slowest approach for Claude-3-5-Sonnet is the Hierarchical framework, with a mean latency of 263.5 seconds.

In the HumanEval task, GPT-4o continues to outperform Claude-3-5-Sonnet in terms of speed. The fastest approach using GPT-4o is the Hierarchical framework, with a mean latency of 38.8 seconds and a minimum of 31.6 seconds. The GPT-4o Hierarchical framework’s performance is also particularly notable for its consistency, boasting a standard deviation of 5.6 seconds. The slowest approach using GPT-4o is the Market framework, a common occurrence, with a mean latency of 94.2 seconds. For

Agent	Framework	Mean	Median	Min	Std. Dev.
GPT-4o	Authoritarian	72.3s	63.0s	42.5s	29.0s
	Market	94.2s	128.2s	16.5s	55.1s
	Hierarchical	38.8s	39.3s	31.6s	5.6s
Claude-3-5-Sonnet	Authoritarian	214.1s	207.8s	196.4s	17.6s
	Market	221.1s	227.5s	189.6s	23.5s
	Hierarchical	266.1s	275.9s	211.3s	41.3s

Table 3.6: Latency results for HumanEval task using GPT-4o and Claude-3-5-Sonnet agents.

Claude-3-5-Sonnet, the Authoritarian framework has the lowest mean latency at 214.1 seconds, while the Hierarchical framework is the slowest with a mean latency of 266.1 seconds.

Overall, the Hierarchical framework is most effective with GPT-4o agents, whereas the Market framework is most effective with Claude-3-5-Sonnet agents. Conversely, the Market framework tends to be slowest with GPT-4o, and the Hierarchical framework is slowest with Claude-3-5-Sonnet.

3.5 Discussion

This study evaluates the performance of three novel prompt optimisation frameworks — Authoritarian, Market-Based, and Hierarchical — when used with GPT-4o and Claude-3-5-Sonnet agents to automatically engineer prompts for GPT-3.5-Turbo across mathematical reasoning (GSM8K), sentiment analysis (SST-2), and code generation (HumanEval). The objective is to investigate multi-agent architectures for automated prompt optimisation and demonstrate the reliability and domain-adaptability of multi-agent frameworks. The frameworks perform directed prompt refinement and explore the extensive space of prompts much more efficiently than a human prompter.

When comparing performance metrics, traditional approaches to prompting (ZS, ZS-CoT and Emotive) generally match or outperform the prompts generated by each framework, irrespective of using GPT-4o or Claude-3-5-Sonnet. For GSM8K, the frameworks struggle to produce better-performing prompts than the ZS Baseline. However, for SST-2 and HumanEval, the Baseline is often matched and at times exceeded. For instance, four out of six framework-agent combinations generate at least one prompt that outperformed the Baseline maximum for HumanEval. The generated prompts are unable to consistently enhance the impressive capabilities of GPT-3.5-Turbo with the traditional prompts; however, the qualitative results of the frameworks underscore their potential to reliably construct sophisticated prompts with complex structures that contain content tailored to the domain at hand.

GPT-4o-driven frameworks consistently perform better than those driven by Claude-3-5-Sonnet. Frame-

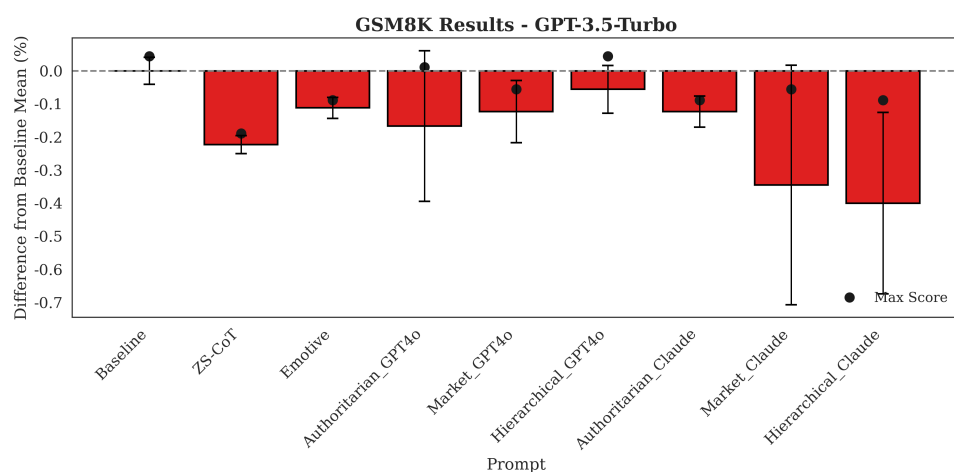
works coupled with GPT-4o also show better performance in terms of lower mean latency in all tasks, highlighting the directness of the GPT-4o agents. When using GPT-4o, the Hierarchical framework appears to be the most efficient, displaying the lowest mean latency and competitive accuracy across all tasks. This efficiency is speculated to be due to the thorough nature of each update step, with the prompt being assessed by multiple agents within each iteration as a result of the tiered architecture. For Claude-3-5-Sonnet, the frameworks are generally evenly matched but often display considerable variance. Both the longer mean run times and higher variance could be a result of the inherent tendency of Claude-3-5-Sonnet to be more creative versus GPT-4o.

The strength of this study lies in its comprehensive comparison across multiple tasks and frameworks, providing a holistic view of multi-agent potential in automated prompt optimisation. One of the main limitations is the number of prompts generated and tested. Due to financial restrictions, only three prompts per framework are evaluated, which likely provides a narrow representation of the frameworks' capabilities. With the frameworks themselves, finding a balance between encouraging exploration and providing guidance is challenging. The core principles offer direction on how to explore the vast prompt space, but define them too rigidly and they may restrict the space too much. The requirement of manually defining the agents and their core principles is also a drawback, as it can be difficult and time consuming depending on the task. The prompt design agents are fixed, but curating domain-specific agents can become challenging for ambiguous or broad tasks. A generic approach to determining agents and core principles is to break down the task in a domain-specific manner to identify possible agents and then determine the key contributions required for each subtask to curate the core principles.

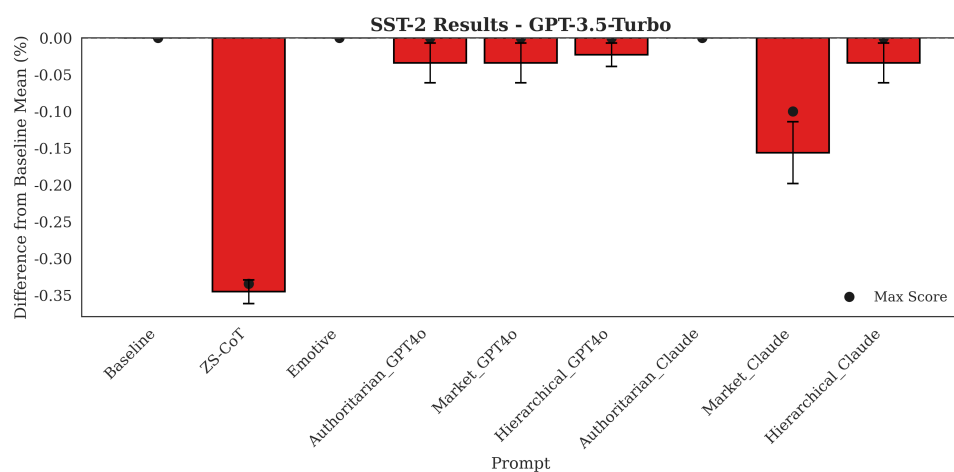
The findings highlight the potential for the frameworks to reliably produce sophisticated and domain-specific prompts. They also highlight key differences in performance between tasks, architecture and agent model used, which can inform future development and deployment strategies for the frameworks. The comparison with traditional approaches underscores the current capabilities of massive third-party models on simple tasks, suggesting that the frameworks may only be worth implementing for more complex tasks; however, additional research is needed. Future research could explore complex tasks as well as how to best define the core principles and how much they affect the diversity of outputted prompts. It would also be beneficial to experiment with approaches to automate agent generation, as an end-to-end process that does not require the manual creation of agents would significantly improve ease of use. This was experimented with but proved difficult to refine to a satisfactory standard in the time constraints of the project.

In conclusion, this study provides valuable insights into the performance of multi-agent architectures

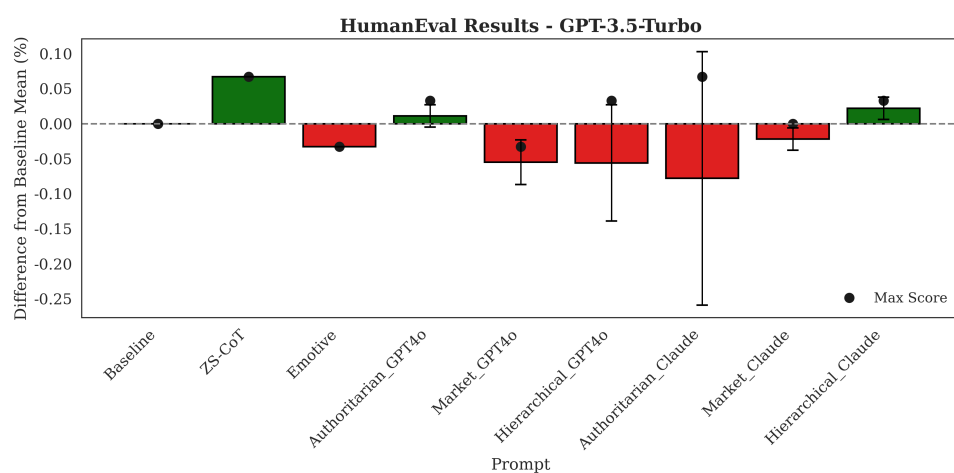
for prompt optimisation. The architectures result in very different final prompts. The simple prompts produced by the Hierarchical framework with GPT-4o are particularly effective for GPT-3.5-Turbo. The comparison with traditional approaches highlights the current limitations of the frameworks in offering performance gains on tasks that are simple for current state-of-the-art models. However, the structure and detail of the prompts and their competitive performance in HumanEval, suggest usefulness for more complex tasks. Overall, the study demonstrates how the multi-agent frameworks can be used to reliably generate domain-specific prompts, but highlights the difficulty of improving the performance of GPT-3.5-Turbo on what are now simple tasks for modern LLMs. Further research into prompt optimisation for complex tasks, the automation of agent generation, and the refinement of agent design are crucial in realising the full potential of these multi-agent frameworks. The studies that follow extend this study by applying the framework-generated prompts to models less capable than GPT-3.5-Turbo and analysing agent implemented strategies that contribute to effective prompting.



(a) GSM8K results with GPT-3.5-Turbo



(b) SST-2 results with GPT-3.5-Turbo



(c) HumanEval results with GPT-3.5-Turbo

Figure 3.4: Visualisation of prompt effectiveness when used with GPT-3.5-Turbo

Chapter 4

Enhancing Open-Source SLMs to Rival Third-Party Models

Third-party LLMs such as OpenAI's GPT-3.5-Turbo are often inaccessible to many users due to their associated costs. This study evaluates the performance of prompts generated by three multi-agent frameworks when applied to two relatively small open-source models: Mistral-V0.3-7B and Llama-3.1-8B. The primary objective is to demonstrate how prompts generated by these prompt optimisation frameworks can enhance the capabilities of accessible open-source models compared to state-of-the-art third-party models that incur inference costs. The study emphasises the utility of these frameworks. The findings reveal marked improvements in mathematical reasoning and sentiment analysis for both the Mistral and Llama models, providing empirical evidence for the effectiveness of these frameworks in optimising prompts for SLMs. The results indicate less consistency in the performance of prompts optimised for code generation.

Building on insights from the formalisation and initial analyses of the Authoritarian, Market-Based and Hierarchical multi-agent frameworks, this study aims to demonstrate how the performance gap between accessible Small Language Models (SLMs) and state-of-the-art LLMs can be bridged by leveraging the prompts generated by the frameworks. The objective is to evaluate the efficacy of framework generated prompts when used by models considerably smaller than GPT-3.5-Turbo, namely Mistral-V0.3-7B and Llama-3.1-8B.

The current literature lacks an evaluation of automated prompt optimisation techniques on widely accessible SLMs, neglecting the practical importance of enhancing SLMs to smaller organisations. Too often capabilities are demonstrated using the best-in-class models which may not be accessible to all and therefore offer little practical value. This study provides perspective on the models evaluated in the literature, before demonstrating the impact the multi-agent frameworks can have on the performance of the 7 and 8 billion parameter Mistral and Llama models mentioned. The hypothesis is that prompts generated using the multi-agent frameworks can significantly improve the performance of the open-source SLMs, enabling them to compete with state-of-the-art LLMs on certain tasks.

The methodology is very similar to that of the previous study and involves a comprehensive evaluation of the prompts formulated by each multi-agent framework. Sets of prompts generated using GPT-4o and Claude-3-5-Sonnet are tested with two open-source SLMs—Mistral-V0.3-7B and Llama-3.1-8B—on mathematical reasoning, code generation and sentiment analysis. The effectiveness of the prompts is assessed using a simple exact match accuracy metric, and by comparing the results to those from the previous study, the aim is to illustrate how the frameworks are particularly beneficial for application with SLMs and determine the extent to which the gap to proprietary models can be reduced by using the framework generated prompts.

Bridging the performance gap between widely accessible open-source SLMs and state-of-the-art third-party LLMs is essential to expand the practical utility of a wider range of models and find cost-effective solutions to practical problems without compromising performance. This study seeks to contribute to this whilst ensuring the versatility and robustness of the multi-agent frameworks in generating prompts for different models.

Through a detailed analysis, this research assesses the effectiveness of the Authoritarian, Market-Based, and Hierarchical frameworks in producing prompts that uplift the performance of relatively weak open-source SLMs. The rest of the study is organised as follows: Section 2 reviews relevant literature; Section 3 outlines the methodology; Section 4 presents the results; and Section 5 concludes with a discussion of the findings.

4.1 Background Literature

The literature review discusses models used in existing literature regarding automated prompt optimisation, before examining Mistral-V0.3 and Llama-3.1 models and the challenges many open-source models face versus proprietary models. It then highlights the importance of bridging the gap in performance between accessible small model variants and state-of-the-art models.

Most existing studies have focused on optimising prompts for LLMs rather than SLMs. For instance, works like [21] and [33] have primarily evaluated their techniques on advanced models such as PaLM 2-L and PaLM 2-S. Similarly, [66] tests on GPT-3.5-Turbo-Instruct, while frameworks such as [51] and [75] have also relied on the capabilities of GPT-3.5-Turbo and InstructGPT, respectively. The only notable exception is [24], which includes evaluations on the smaller Alpaca-7B model alongside GPT-3.5. This trend highlights a significant gap in the literature, where the emphasis has been predominantly on LLMs, leaving the potential of automated prompt optimisation for SLMs like Mistral-V0.3-7B and Llama-3.1-

8B relatively unexplored. Addressing this is crucial for user without the means to access third-party models and who need to extract the best performance possible from much smaller open-source models.

The open-source Llama-3 and Mistral-V0.3 models offer notable capabilities in various applications. However, they face significant challenges when compared to proprietary models. Llama-3 is designed to handle multiple languages and integrates image, video, and speech capabilities, positioning it competitively against models like GPT-4 [19]. It's 70B specification has also demonstrated strong abilities in code generation tasks, excelling in benchmarks such as HumanEval and MBPP through innovative training techniques [20]. Mistral-V0.3 incorporates innovative fine-tuning techniques that enhance its performance with fewer parameters, making it a cost-effective alternative to larger proprietary models [45]. The model improves upon natural language generation (NLG) tasks through a novel evaluation framework that outperforms previous benchmarks, showcasing its reasoning capabilities [45]. Despite their capabilities, the best and largest versions of both models require significant computational resources [68], and while competitive, both often fall short of proprietary models like GPT-4 in specific tasks, particularly in complex reasoning [1]. This is the case even more so for smaller model variants including Mistral-V0.3-7B and Llama-3.1-8B.

The largest versions of Llama 3 and Mistral V0.3 models showcase impressive capabilities in language processing and generation, yet they face significant challenges regarding resource demands and performance in complex tasks when juxtaposed with proprietary models. Enhancing the practical application of smaller model variants across the landscape is essential for those with limited resources. Research into prompt engineering and automated prompt optimisation is crucial to help address these challenges, providing cost-effective solutions for high quality output.

4.2 Methodology

This study focuses on evaluating the performance of prompts generated by different multi-agent architectures when used by Mistral-V0.3-7B and Llama-3.1-8B. These models are selected because they represent two general-purpose and widely accessible open-source SLMs, capable of performing inference at a reasonable speed on consumer-grade hardware such as an Apple M1 chip with 16GB of memory. While the overall methodology is similar to that of the previous study, this section highlights the subtle differences in the evaluation process due to the use of locally hosted open-source SLMs and reiterates the general experimental procedure.

4.2.1 Evaluating Locally Hosted Models

For performing inference with open-source models various approaches exist including accessing models via platforms like HuggingFace or Ollama. Both offer easy ways to download and run open-source models locally. Ultimately, Ollama is chosen for this study due to its compatibility with LangChain, ease of implementation, and the ability to download and run models locally without requiring CUDA support. Since the study uses small sample sizes for evaluation, access to large-scale computing resources is unnecessary. To ensure efficiency, timeout measures are implemented for the locally hosted models to handle cases where inference is taking a substantial amount of time.

4.2.2 Experimental Procedure

The experimental procedure mirrors that of the previous study, but with a focus on open-source SLM performance improvements rather than GPT-3.5-Turbo. Mistral-V0.3-7B and Llama-3.1-8B are used in place of GPT-3.5-Turbo, however, the results from GPT-3.5-Turbo are considered in the analysis in order to establish the extent to which the open-source models can compete with a large proprietary model. The temperature of all models is set to 0.0 as with the previous study.

GSM8K, SST-2 and HumanEval are again used for testing on a diverse set of tasks, and the results between Mistral and Llama models compared to one another, as well as against the results of the previous study. The same set of prompts are tested for consistency and validity, and as such the optimisation processes are not re-run, saving additional financial expense. Again the Wilcoxon Signed-Rank Test is performed to establish the significance of results.

4.3 Results

The results of study two are split by inference model. First, the results using Mistral-V0.3-7B are detailed, followed by the results using Llama-3.1-8B. For each model, identical prompts to the previous study are used (see appendix C). The results are presented by dataset with the GPT-3.5-Turbo results being used as the third-party LLM comparison. Key differences in the results between prompts are highlighted, as well as differences in the effectiveness for each model.

4.3.1 Mistral-V0.3-7B

In contrast to GPT-3.5-Turbo, the 7 billion parameter Mistral-V0.3 model possesses relatively weak performance with simple prompts across GSM8K, SST-2 and HumanEval. The results below demonstrate

the effectiveness of using the multi-agent optimisation frameworks to refine the baseline prompt.

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	20.0%	20.0%	20.0%	0.0%
	ZS-CoT	26.7%	26.7%	26.7%	0.0%
	Emotive	6.7%	6.7%	6.7%	0.0%
GPT-4o	Authoritarian	14.4%	3.3%	40.0%	18.1%
	Market	43.3%	43.3%	53.3%	8.2%
	Hierarchical	36.7%	40.0%	46.7%	9.8%
Claude-3-5-Sonnet	Authoritarian	27.8%	30.0%	53.3%	21.8%
	Market	23.3%	23.3%	46.7%	19.1%
	Hierarchical	6.7%	0.0%	20.0%	9.4%

Table 4.1: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for GSM8K with Mistral-V0.3-7B

Table 3.1 summarises the performance of prompts across the GSM8K task. In the GSM8K task, ZS-CoT prompts improve performance from 20.0% to 26.7%, while Emotive prompts perform poorly at 6.7%. When using GPT-4o agents, the Market-Based framework produces the best performing prompts, massively improving accuracy from 20.0% to 43.3%, with a peak accuracy of 53.3%. The Authoritarian framework produces the worst performing prompts the GPT-4o powered frameworks. For Claude-3-5-Sonnet agents, the Authoritarian framework produces the highest performing prompts achieving an average accuracy of 27.8% and a maximum of 53.3%, improving over the baseline of 20.0%.

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	66.7%	66.7%	66.7%	0.0%
	ZS-CoT	70.0%	70.0%	70.0%	0.0%
	Emotive	71.1%	70.0%	73.3%	0.0%
GPT-4o	Authoritarian	86.7%	86.7%	90.0%	2.7%
	Market	71.1%	73.3%	83.3%	11.0%
	Hierarchical	81.1%	83.3%	93.3%	11.0%
Claude-3-5-Sonnet	Authoritarian	48.9%	53.3%	80.0%	27.4%
	Market	43.3%	60.0%	66.7%	28.4%
	Hierarchical	70.0%	66.7%	80.0%	7.2%

Table 4.2: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for SST-2 with Mistral-V0.3-7B

Table 3.2 presents the results for the SST-2 task. For SST-2, ZS-CoT prompts show a slight improvement over the baseline (66.7% to 70.0%), and Emotive prompts perform similarly. When using GPT-4o agents, the Authoritarian framework again excels, with prompts achieving an average accuracy of 86.7% and a maximum of 90.0%; a substantial increase from the Baseline of 66.7%. The Hierarchical framework produces the single best-performing prompt achieving 93.3% accuracy, but lacks consistency exhibiting high variance. For Claude-3-5-Sonnet agents, the Hierarchical framework produces prompts that achieves

the highest average accuracy at 70.0%, surpassing the improvements seen with Emotive prompting. An impressive 80.0% is achieved by two standalone prompts from the Authoritarian and Hierarchical frameworks.

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	46.7%	46.7%	46.7%	0.0%
	ZS-CoT	43.3%	43.3%	43.3%	0.0%
	Emotive	46.7%	46.7%	46.7%	0.0%
GPT-4o	Authoritarian	45.5%	50.0%	53.3%	8.8%
	Market	35.6%	40.0%	46.7%	11.3%
	Hierarchical	38.9%	36.7%	46.7%	5.7%
Claude-3-5-Sonnet	Authoritarian	25.5%	23.3%	40.0%	11.0%
	Market	38.9%	40.0%	46.7%	6.9%
	Hierarchical	42.2%	46.7%	46.7%	6.3%

Table 4.3: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for HumanEval with Mistral-V0.3-7B

Table 3.3 presents the results for the HumanEval task. In the HumanEval task, both Baseline and ZS-CoT prompts achieve an accuracy of 46.7%. When using GPT-4o agents, the Authoritarian framework performs best, producing prompts that achieve an average of 45.5%, with individual prompts exceeding the baseline of 46.7% on multiple occasions. For Claude-3-5-Sonnet agents, the Hierarchical framework produces prompts displaying the best performance with an average accuracy of 42.2%, just below the 46.7% displayed by the Baseline. However, both Market-Based and Hierarchical frameworks prompts achieve a maximum of 46.7% which equals the Baseline and Emotive prompt performance.

Frameworks coupled with GPT-4o agent consistently outperform those coupled with Claude-3-5-Sonnet agents across GSM8K, SST-2, and HumanEval. The Market-Based framework with GPT-4o agents produces prompts that excel in GSM8K with a maximum accuracy of 53.3%, while those from the Hierarchical framework leads in SST-2 with a maximum accuracy of 93.3%. Prompts resulting from the Authoritarian framework achieve a respectable 53.3% in HumanEval. When using Claude-3-5-Sonnet agents, the best performing prompts come from the Authoritarian framework in GSM8K (53.3%), and the Hierarchical framework in SST-2 and HumanEval (80.0% and 46.7% respectively). Overall, GPT-4o agents, particularly within the Market-Based and Authoritarian frameworks, demonstrate the highest efficacy across tasks for Mistral-V0.3-7B.

Remarkably, the framework generated prompts for SST-2 result in comparable scores to GPT-3.5-Turbo’s best performance. The 93.3% achieved by a prompt from the Hierarchical framework with GPT-4o is only 3.4% below the peak performance of GPT-3.5-Turbo. The best performances of Mistral-V0.3-7B on the GSM8K and HumanEval are notably lower than the best performances observed with GPT-3.5-Turbo.

4.3.2 Llama-3.1-8B

Similarly to Mistral-V0.3-7B, Llama-3.1-8B demonstrates considerably lower performance than GPT-3.5-Turbo with the same traditional prompts. Using prompts optimised by the multi-agent frameworks frequently results in considerable improvements.

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	3.3%	3.3%	3.3%	0.0%
	ZS-CoT	10.0%	10.0%	10.0%	0.0%
	Emotive	6.7%	6.7%	6.7%	0.0%
GPT-4o	Authoritarian	33.3%	30.0%	66.7%	26.0%
	Market	20.0%	13.3%	43.3%	17.0%
	Hierarchical	4.4%	6.7%	6.7%	5.7%
Claude-3-5-Sonnet	Authoritarian	27.8%	26.7%	56.7%	23.1%
	Market	10.0%	13.3%	16.7%	7.2%
	Hierarchical	10.0%	3.3%	26.7%	11.9%

Table 4.4: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet Approaches for GSM8K with Llama-3.1-8B

Table 3.1 summarises the performance of prompts across the GSM8K task. In the GSM8K task, ZS-CoT prompts improve performance from 3.3% to 10.0%, while Emotive prompts perform only marginally better than the Baseline (6.7%). When using GPT-4o agents, the Authoritarian framework produces prompts that exhibit the best performance, notably improving accuracy from 3.3% to 33.3%, with a peak accuracy of 66.7%. The Authoritarian framework also produces the highest performing prompts when using Claude-3-5-Sonnet, achieving an average accuracy of 27.8% and maximum of 56.7%.

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	80.0%	80.0%	80.0%	0.0%
	ZS-CoT	81.1%	80.0%	83.3%	0.0%
	Emotive	80.0%	80.0%	80.0%	0.0%
GPT-4o	Authoritarian	88.9%	90.0%	96.7%	6.8%
	Market	85.6%	80.0%	96.7%	7.9%
	Hierarchical	93.3%	96.7%	96.7%	4.7%
Claude-3-5-Sonnet	Authoritarian	80.0%	86.7%	96.7%	17.0%
	Market	32.2%	23.3%	66.7%	25.3%
	Hierarchical	86.7%	90.0%	90.0%	4.7%

Table 4.5: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for SST-2 using Llama-3.1-8B

Table 3.2 presents the results for the SST-2 task. For SST-2, ZS-CoT prompts show a slight improvement over the Baseline average (80.0% to 81.1%), and Emotive prompts perform similarly. When using GPT-4o agents, prompts from the Hierarchical framework excel, achieving an average accuracy of 93.3%

and a maximum of 96.7%. The Authoritarian frameworks also produces prompts that perform well, achieving an average accuracy of 88.9% and a maximum of 96.7%. For Claude-3-5-Sonnet agents, the Hierarchical framework produces prompts achieving the highest average accuracy at 86.7%, surpassing the improvements seen with Emotive prompting. An impressive 96.7% is achieved by prompts from the Authoritarian framework, but the prompts display high variance. Notably poor performance is exhibited by the Market-Based framework prompts.

Approach	Prompt Type	Mean	Median	Max	Std Dev
Traditional	Baseline	60.0%	60.0%	60.0%	0.0%
	ZS-CoT	66.7%	66.7%	66.7%	0.0%
	Emotive	66.7%	66.7%	66.7%	0.0%
GPT-4o	Authoritarian	33.3%	23.3%	60.0%	19.0%
	Market	62.2%	66.7%	70.0%	8.7%
	Hierarchical	57.8%	56.7%	73.3%	12.3%
Claude-3-5-Sonnet	Authoritarian	34.5%	40.0%	46.7%	12.9%
	Market	61.1%	60.0%	66.7%	4.1%
	Hierarchical	55.6%	60.0%	60.0%	6.3%

Table 4.6: Comparison of traditional, GPT-4o, and Claude-3-5-Sonnet approaches for HumanEval with Llama-3.1-8B

Table 3.3 presents the results for the HumanEval task. In the HumanEval task, both ZS-CoT and Emotive prompts achieve 66.7%. When using GPT-4o agents, the Market-Based framework produces the best performing prompts, with them achieving an average accuracy of 62.2% and a maximum of 70.0%. The Hierarchical framework also produces prompts displaying strong performance with an average accuracy of 57.8% and a maximum of 73.3%. With Claude-3-5-Sonnet agents, the Market-Based framework again produces prompts displaying the best performance with an average accuracy of 61.1% and maximum 66.7%, both surpassing the Baseline performance.

Similarly to the Mistral-V0.3-7B results, GPT-4o agent frameworks produce prompts that consistently outperform those using Claude-3-5-Sonnet when used with Llama-3.1-8B across GSM8K, SST-2, and HumanEval tasks. The Authoritarian framework with GPT-4o excels in GSM8K with a mean performance of 66.7%, while those from the Hierarchical framework are best in SST-2 and HumanEval with maximum scores of 96.7% and 73.3% respectively. When using Claude-3-5-Sonnet, the best prompts came from the Authoritarian framework in GSM8K (33.3%), the Hierarchical framework in SST-2 (93.3%) and the Market-Based framework in HumanEval (61.1%), although these were generally below GPT-4o framework prompt performances.

Once again, the framework generated prompts for SST-2 resulted scores that competed with GPT-3.5-Turbo’s best performance. Each framework when used with GPT-4o generated a prompt that achieved

96.7% accuracy which matches the peak performance of GPT-3.5-Turbo. The performance of Llama-3.1-8B on HumanEval with a Hierarchical framework generated prompt was only 13.4% below that of GPT-3.5-Turbo, however the performance on GSM8K remained considerably worse.

In neither the Mistral-V0.3-7B or Llama-3.1-8B cases did the Wilcoxon Signed-Rank Test suggest significant improvements were achieved using the framework generated prompts. However, as previously stated, the sample sizes of three make it very difficult to comment on the significance of the mean results. It is evident from the raw statistic that huge improvements in performance result from using the frameworks.

4.4 Discussion

This study extends the previous study by evaluating the effectiveness of prompts produced by the multi-agent frameworks when used by smaller open-source models on tasks such as mathematical reasoning, sentiment analysis, and code generation. The objective is to demonstrate the capacity of these frameworks to bridge the gap between widely accessible open-source models and third-party models that incur inference costs. The findings underscore the effectiveness of the frameworks in refining prompts for both Mistral-V0.3-7B and Llama-3.1-8B models, resulting in significant performance improvements. Variability in performance of different frameworks and agent models across tasks illustrates the responsiveness of different models to different prompts.

The optimisation frameworks were particularly effective when used with GPT-4o to develop prompts for mathematical reasoning and sentiment analysis. Performance improvements of up to 33.3% with Mistral-V0.3-7B and 63.4% with Llama-3.1-8B were observed on the GSM8K dataset. Improvements on the SST-2 dataset were also substantial, but less pronounced. The HumanEval task proved more challenging, particularly for Llama-3.1-8B, where Emotive and ZS-CoT prompting outperformed most other prompts. This contrasts with the results when using GPT-3.5-Turbo, where the frameworks were most influential in prompt refinement for code generation.

The results indicate considerable variation in the performance of different framework-agent combinations across tasks, making it difficult to establish definitive guidelines for optimal pairings. For instance, prompts produced by the Hierarchical framework with Claude-3-5-Sonnet demonstrated competitive performance across code generation and sentiment analysis with both Mistral and Llama models, but struggled with mathematical reasoning when used with Mistral-V0.3-7B exhibiting performance over 20% lower than those generated by other frameworks. A similar pattern was observed for the Hierarchical

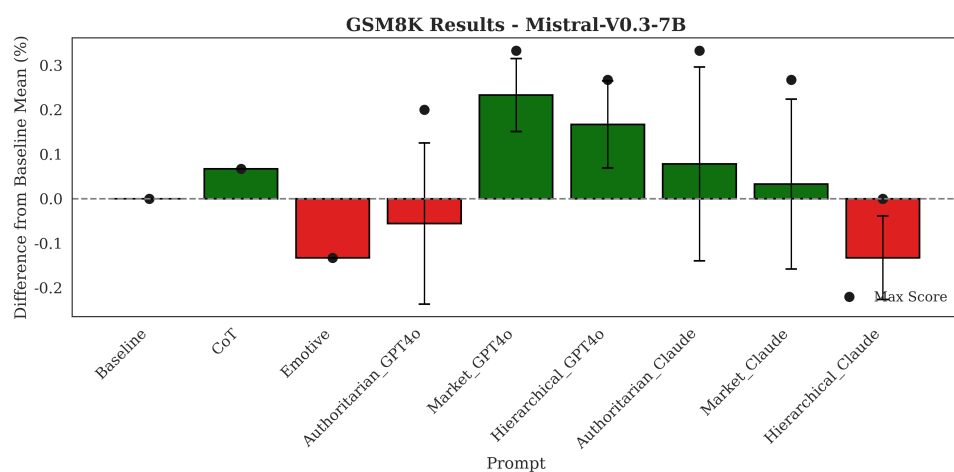
framework with GPT-4o, where prompts performed well except for those engineered for GSM8K and applied with Llama-3.1-8B, showing a 10% performance deficit compared to other frameworks. The gap in performance to GPT-3.5-Turbo is considerably reduced for both models, and even closed on the sentiment analysis task; however, there remains notable differences for mathematical reasoning and code generation.

The noticeable disparity in performance between GPT-4o and Claude-3-5-Sonnet agents was unexpected, given that both are state-of-the-art LLMs with exceptional natural language understanding capabilities. This discrepancy was particularly pronounced under the Hierarchical framework in the GSM8K task, suggesting that GPT-4o may be more effective in structured environments. This is reflected in the direct and disciplined nature of the prompts it generates, which rarely offer creative twists. However, upon inspection of the raw outputs, it became clear that the root of this disparity lies in the formatting of outputs generated by the Claude-3-5-Sonnet prompts, with Mistral-V0.3-7B and Llama-3.1-8B models seemingly ignoring the instructions provided and the regular expression process during evaluation unable to extract answers properly.

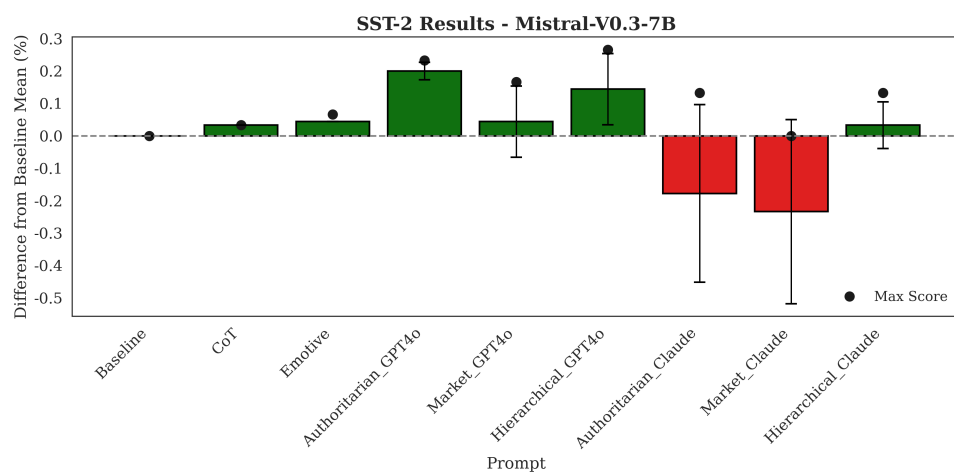
Despite providing strong evidence for the effectiveness of the frameworks in automating prompt optimisation for SLMs, the study possesses limitations. One notable issue is the reliance on an exact output format, which resulted in frequent false negative results during evaluation. Particularly with Claude-3-5-Sonnet prompts engineered for GSM8K, the correct answer was often outputted but parsed incorrectly or not at all due to non-adherence to strict output format instructions. To accommodate false negatives a more flexible evaluation metric such as fuzzy matching could be used. However, the results as they are also signify a failure in the prompts generated using Claude-3-5-Sonnet since they unsuccessfully enforce the output instruction. This issue was less prevalent with GPT-4o-generated prompts, where output format instructions were more frequently placed towards the end of the prompt, as opposed to the middle as done by Claude-3-5-Sonnet agents.

In conclusion, this study demonstrates the potential of multi-agent prompt optimisation frameworks to dramatically enhance the performance of smaller open-source models, such as Mistral-V0.3-7B and Llama-3.1-8B. The findings highlight the gap in performance between open-source SLMs and third-party LLMs can be remarkably reduced by using the multi-agent approaches to automate prompt optimisation. There is clear variability in performance across different frameworks and agent models, with notable success when using GPT-4o. However, the reliance on specific output formats in the evaluation process led to regular false negatives, especially with Claude-3-5-Sonnet prompts, blemishing results. These insights underscore the need for an improved evaluation procedure, but do not detract from the impressive

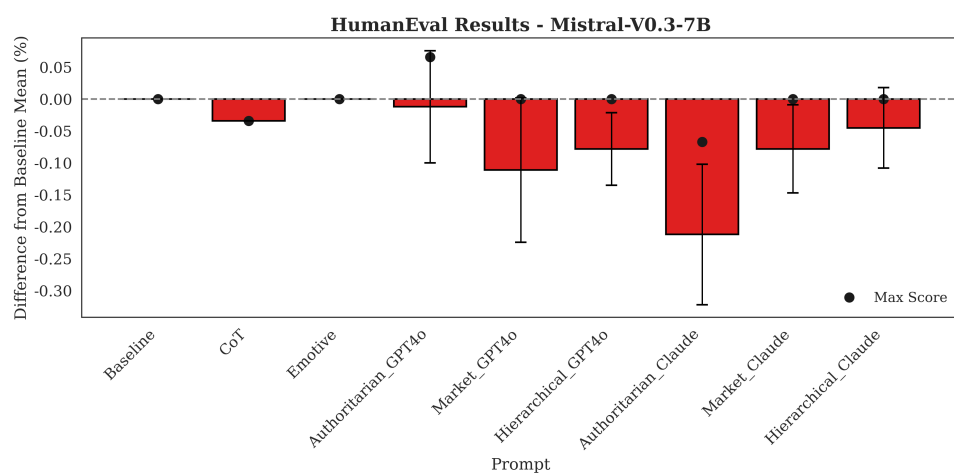
impact the frameworks can have for SLMs. The next study explores aspects of high performing prompts and the conversation that resulted in them in more depth.



(a) GSM8K results with Mistral-V0.3

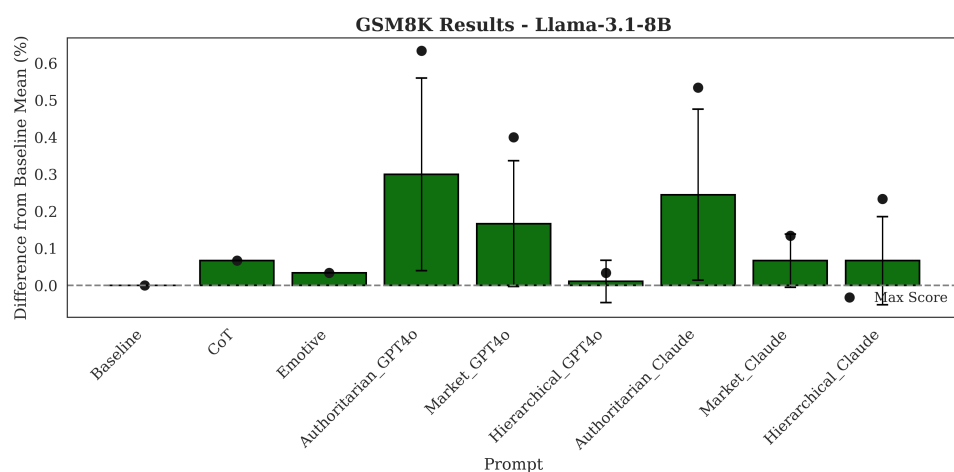


(b) SST-2 results with Mistral-V0.3

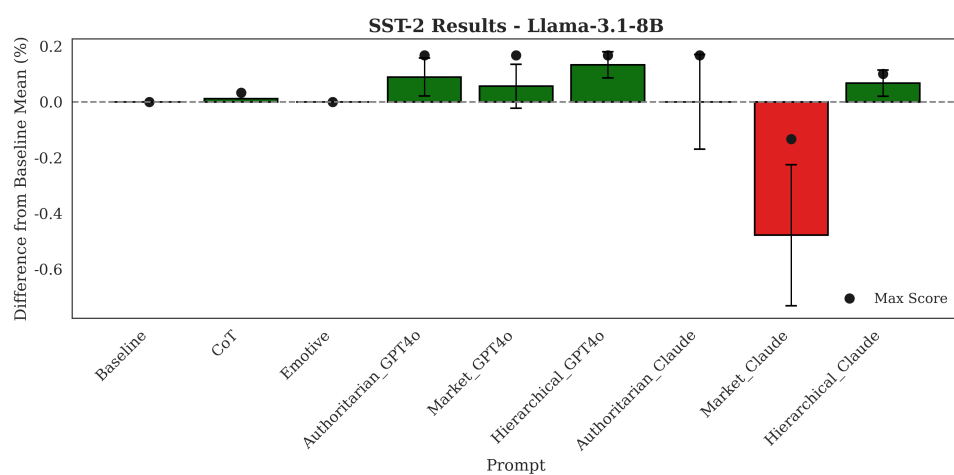


(c) HumanEval results with Mistral-V0.3

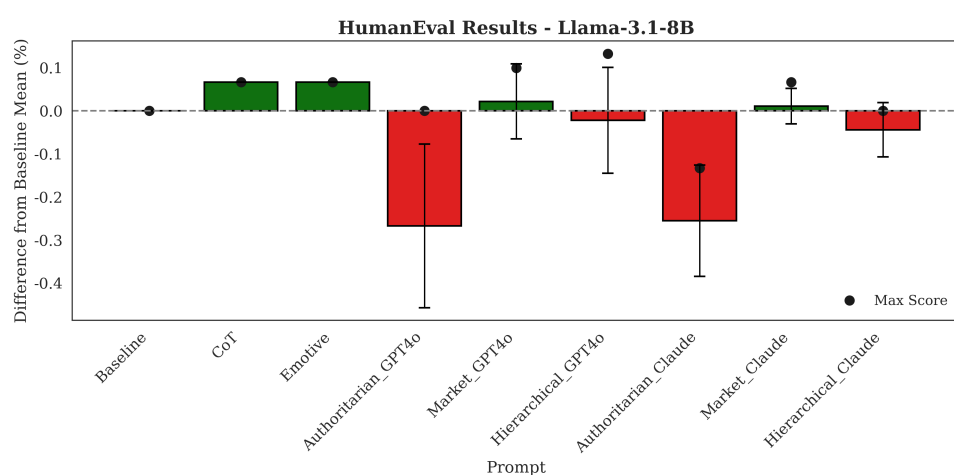
Figure 4.1: Visualisation of prompt effectiveness when used with Mistral-V0.3



(a) GSM8K results with Llama-3.1-8B



(b) SST-2 results with Llama-3.1-8B



(c) HumanEval results with Llama-3.1-8B

Figure 4.2: Visualisation of prompt effectiveness when used with Llama-3.1-8B

Chapter 5

Agent Interactions for High-Performing Prompt Generation

This final study investigates the conversational interactions between agents involved in generating high-performing prompts. By analysing both the conversation logs and the resulting prompts that lead to performance improvements, the research aims to uncover the techniques and strategies that agents use to create effective prompts. The focus is on understanding how different models—specifically GPT-4o and Claude-3-5-Sonnet—employ distinct strategies within each framework, and how effective prompts differ between tasks and inference models. Existing prompt engineering techniques are often based on strategies that are effective in human-to-human communication, but these methods may not be optimal when interacting with LLMs, therefore the analysis provides an alternative perspective on effective prompt generation. The key findings indicate that effective strategies vary significantly depending on the architecture and LLM used for optimisation. Notably, the Market-Based architecture often resorts to contextual guidance and examples to produce effective prompts, whereas the Authoritarian framework focuses on directness with well defined-instructions. It is also evident that GPT-4o tends to produce more direct and focused prompts, while Claude-3-5-Sonnet exhibits greater creativity in its approach utilising techniques not often encouraged by human prompters. The study speculates that aspects of all approaches could be synergised to provide a standardised approach to quality prompt generation.

5.1 Introduction

This study investigates the intricate dynamics of agent interactions within multi-agent frameworks, aiming to uncover the strategies that agents employ to improve prompts. By analysing the conversations and interactions among agents that lead to the production of the most effective prompts, the research seeks to identify general, model- and task-specific strategies that contribute to high-performing prompt generation.

Unlike prior studies that have predominantly focused on the structural aspects and comparative per-

formance of the different frameworks, this study centers on the qualitative aspects of agent behavior. Understanding how agents collaborate, deliberate, and refine prompts can provide valuable insights into best practices for prompt engineering, as well as suggestions for refining the frameworks themselves. The analysis involves a detailed examination of conversation logs within these frameworks, alongside the evaluation of the final prompts produced.

The literature review underpinning this study primarily focuses on established best practices for prompting derived from human expertise. While previous studies have explored prompt optimisation through human logic and iterative processes, this research approaches prompt construction from a novel perspective, examining how LLM agents construct and refine prompts autonomously. By analysing the implementations developed by agents during their interactions, the study aims to help standardise prompt engineering practices by reinforcing existing techniques and identifying new ones.

Methodologically, this research involves capturing and analysing comprehensive conversation logs from each of the three multi-agent frameworks that produce high-performing prompts. The study examines outputs from each framework using GPT-4o and Claude-3-5-Sonnet agents, acknowledging their differing styles and approaches and which are most effective for a selection of tasks and models. By surveying these logs and the corresponding final prompts, and applying rigorous qualitative analysis, the research seeks to pinpoint the key elements that contribute to successful prompt engineering.

The motivation for this study stems from the need for a standardised approach to prompts and the observation that current best practices in prompt engineering are largely based on human intuition and logic. This research offers a unique perspective by exploring how agents, rather than humans, interact to optimise prompts, and how these interactions can be harnessed to generate superior prompt outputs. The primary objective is to provide a comprehensive understanding of the best practices employed by agents within each framework. By identifying and analysing effective interaction patterns, the study aims to contribute to the development of a standardised approach to prompting with advanced and efficient prompt engineering techniques. Additionally, the findings may offer pathways for enhancing the performance of the three frameworks under investigation.

The remainder of this section is structured as follows: Section 2 reviews relevant literature; Section 3 outlines the methodology; Section 4 presents the results; and Section 5 concludes with a discussion of the findings.

5.2 Background Literature

The literature review begins by detailing current challenges and biases in prompt engineering and the best practices implemented to overcome them, before indicating areas addressed by the work in this study.

Crafting effective prompts remains challenging due to several factors. One significant barrier is the expectations that arise from human-to-human instructional experiences, which can lead to over-generalisation in prompt design [44, 74]. This tendency to rely on preconceived notions can hinder the performance of LLMs, as prompts must be tailored specifically to the unique capabilities and limitations of these models. Moreover, the complexity of language and the nuances involved in communication necessitate careful tuning of words in prompt design. Researchers emphasise that avoiding laborious prompt engineering through ingenious design can significantly enhance user experience and model performance.

To address the challenges associated with prompt design, several best practices have been identified. One effective strategy is to utilise structured prompts that include specific subject and style keywords. This approach not only guides the model in generating coherent outputs but also reduces ambiguity [74]. Additionally, chaining LLM steps together has been shown to improve the quality of task outcomes while enhancing the transparency and controllability of the interaction [37].

While current studies emphasise structured prompts, there is a lack of comprehensive guidelines on the optimal structure and elements necessary for various applications. Previous research also exhibits a bias towards human-to-human interactions which may not be optimal for LLMs. Investigating how humans and LLMs can collaboratively design effective prompts is a promising direction to address this, and this study begins to explore it. Contributions to a robust standardised framework are essential for guiding future research and development.

5.3 Methodology

This study builds upon the previous studies, utilising data collected during the creation of prompts. To conduct an in-depth analysis of agent interactions, conversation logs from the highest-performing prompts are reviewed. This section outlines the contents and creation process of these conversation logs and details the methodology employed to extract and analyse relevant information.

5.3.1 Conversation Logs

The conversation logs consist of sequences of messages exchanged between agents during the process of refining a prompt. These logs capture the message history throughout the optimisation process and are extracted upon completion. The logs begin with the user’s initial message (Baseline prompt) and continue with alternating exchanges between agents. The content of these messages includes decisions made by the agents, as well as the justifications and thought processes behind these decisions.

5.3.2 Experimental Procedure

The conversation logs are saved as JSONL files, with separate files created for each model and task (e.g., GSM8K, SST-2, and HumanEval for Mistral, Llama, and GPT-3.5-Turbo). This structure allows for the development of model- and task-specific techniques. The study focuses exclusively on conversation logs that resulted in high-performing prompts, as the objective is to identify effective prompting strategies. A high-performing prompt is defined as one that meets or exceeds the baseline.

The files are analysed by a human evaluator and a carefully designed LLM agent. The LLM agent is powered by GPT-4o and equipped with tools to access and read the JSONL files, and it is tasked with extracting information regarding similarities and differences between the conversations and their outcomes. The agent is instructed to focus on fine details that might be overlooked by the human evaluator.

Within the context of each task, the GPT-4o agent first identifies similarities across all conversations and prompts, including commonly utilised agents and frequently employed techniques. The agent then highlights differences in decisions and outcomes between frameworks, particularly when using GPT-4o versus Claude-3-5-Sonnet. The aim is to establish techniques that are specific to individual frameworks and agent models. After extracting key information, the results are collated and reviewed. A set of task-specific and general best practices is established for each inference model.

5.4 Results

The results of the conversation log analysis are broken down by inference model. First, the log extractions of the best performing prompts with GPT-3.5-Turbo are presented, followed by those for Mistral-V0.3-7B and Llama-3.1-8B. The results detail similarities between the high performing prompts as well the distinct actions implemented by each framework and GPT-4o and Claude-3-5-Sonnet.

5.4.1 High Performing Prompts for GPT-3.5-Turbo

The results for GPT-3.5-Turbo do not include GSM8K since no prompts from the frameworks performed highly enough.

SST-2

Common Strategies Across Frameworks and Models:

- **Conciseness and Clarity:** Emphasis on making prompts straightforward by removing unnecessary words and complex instructions.
- **Contextual Relevance:** Providing context or a role, such as framing it as a sentiment analyst or a bot working for a company.
- **Step-by-Step Instructions:** Breaking down the task into clear, incremental steps to guide the model through the analysis process.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Aspect-based analysis and detailed feedback loops	Immersive scenarios, multi-step processes, and graded sentiment analysis

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Emphasises direct and clear instructions, task alignment, and contextual relevance
Market	Focuses on aspect-based and contextual analysis, includes examples to guide the model
Hierarchical	Utilises incremental prompting, role assignment, and examples to enhance the model’s understanding and performance

HumanEval

Common Strategies Across Frameworks and Models:

- **Clarity and Conciseness:** Emphasis on making the prompt clear and concise.

- **Explicit Instructions:** Ensuring the instructions are explicit and specify the task clearly.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Providing examples and demonstrations	Professional and technical tone, step-by-step instructions, and contextual relevance

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Strict adherence to guidelines and task alignment, ensuring no modifications to existing restrictions
Hierarchical	Structured approach, emphasising efficiency and elegance, guiding the model through the implementation

5.4.2 High Performing Prompts for Mistral-V0.3-7B

GSM8K

Common Strategies Across Frameworks and Models:

- **Clarity and Conciseness:** Consistent emphasis on making the prompts clear and concise.
- **Step-by-Step Instructions:** Incorporating step-by-step instructions to guide the LLM through the problem-solving process.
- **Examples:** Providing examples to illustrate the expected output and problem-solving process.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Clarity, directness, and providing examples	Detailed steps, making initial estimates, considering multiple approaches

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Directness and clarity, with detailed step-by-step instructions and examples
Market	Role assignment, incremental prompting, breaking down tasks into smaller steps, providing creative tips
Hierarchical	Structured approach with clear steps, emphasising mathematical soundness and verification of solutions

SST-2

Common Strategies Across Frameworks and Models:

- **Task Clarity:** Ensuring the task of classifying sentiment as either positive or negative is clearly stated.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Clarity and conciseness, reiterating the task	Detailed instructions and examples, metaphors

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Comprehensive guidelines on sentiment analysis, reiterating the task and output format
Market	Emphasises context, nuances, and graded sentiment
Hierarchical	Structured approaches, breaking down text into aspects, evaluating each aspect's sentiment

HumanEval

Common Strategies Across Frameworks and Models:

- **Clarity of Task:** Clear instructions to complete a function based on its signature and docstring.

- **Explicit Instructions:** Including explicit instructions on using the information from the signature and docstring to infer the function’s logic.
- **Formatting:** Improving the readability and structure of the prompts.
- **Role Design:** Assigning the model a specific role to help it understand the task better.
- **Contextual Examples:** Providing contextually relevant examples to ensure the model can handle various scenarios.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Core principles like scalability and maintainability, considering trade-offs	Incremental prompting, visualization, and self-reflection, unique steps like creating an ASCII flowchart

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Strict adherence to guidelines and success criteria, explicit mention of core principles
Market	Analysing the function signature and docstring, planning the implementation, incrementally developing the function
Hierarchical	Role-based instructions, checking for logical consistency, handling edge cases, best coding practices

5.4.3 High Performing Prompts for Llama-3.1-8B

GSM8K

Common Strategies Across Frameworks and Models:

- **Clarity and Conciseness:** Consistent emphasis on making the prompt clear and concise.
- **Step-by-Step Instructions:** Incorporating a step-by-step approach to solving the problem.
- **Examples:** Providing examples to illustrate the expected process and output.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Direct language, task alignment, examples	Mathematical rigor, creative approaches, visualisation

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Very detailed steps, encouraging multiple approaches and double-checking
Market	Specific roles, contextual reminders, examples for clarity
Hierarchical	Structured approach, engaging tone, task-specific guidance

SST-2**Common Strategies Across Frameworks and Models:**

- **Task Clarity:** Emphasising the importance of clearly instructing the LLM to classify the sentiment of the text as either positive or negative.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Reiterating task requirements, ensuring unsimplicity	Conciseness and clarity, removing unnecessary words

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Rigid approach, strict adherence to task requirements and output format
Market	Emphasising context and nuances, suggesting additional instructions and examples
Hierarchical	Breaking down tasks into clear steps, detailed guidance, contextual relevance, incremental prompting

HumanEval

Common Strategies Across Frameworks and Models:

- **Explicit Instructions:** Emphasising providing clear and explicit instructions to guide the model in completing the function.
- **Best Practices:** Adhering to best practices in software engineering.
- **Incremental Development:** Breaking down the task into smaller, manageable steps.
- **Documentation and Comments:** Including clear and concise comments to explain complex logic or important steps.
- **Self-Review and Testing:** Encouraging self-review and testing of the implementation with various input scenarios.

Differences Between GPT-4o and Claude-3-5-Sonnet:

GPT-4o	Claude-3-5-Sonnet
Core principles like scalability and maintainability, considering trade-offs	Incremental prompting, visualisation, and self-reflection, unique steps like creating an ASCII flowchart

Differences Between Frameworks:

Framework	Characteristics
Authoritarian	Strict adherence to guidelines and success criteria, explicit mention of core principles
Market	Analysing the function signature and docstring, planning the implementation, incrementally developing the function
Hierarchical	Role-based instructions, checking for logical consistency, handling edge cases, best coding practices

5.4.4 Summary of Differences

Differences Between Models

Differences in Strategies
• GPT-3.5-Turbo thrives with simplicity, clarity, and contextual relevance, often using step-by-step guidance. • Mistral-V0.3-7B and Llama-3.1-8B also benefit from clarity and step-by-step instructions but are particularly responsive to examples.

Differences Between Frameworks

Differences in Strategies
• Authoritarian frameworks focus on direct, clear instructions with strict task alignment and detailed steps. • Market frameworks provide creative, contextual guidance, focusing on aspect-based analysis and using examples. • Hierarchical frameworks use structured, incremental approaches, often assigning roles to enhance understanding.

Differences Between Agents

Differences in Strategies
• GPT-4o uses direct language, examples, and structured feedback, focusing on clarity and task engagement. • Claude-3-5-Sonnet adopts immersive scenarios, step-by-step instructions, and creative, reflective approaches.

Differences Between Tasks

Differences in Strategies

- **GSM8K** benefits from explicit step-by-step instructions and examples to solve mathematical problems effectively.
- **SST-2** often requires role assignment to focus on sentiment analysis, with emphasis on task clarity and contextual relevance.
- **HumanEval** needs detailed instructions, examples, and documentation, often involving role assignments like software engineer.

5.5 Discussion

This study analyses the conversation logs of high-performing prompts for GPT-3.5-Turbo, Mistral-V0.3-7B, and Llama-3.1-8B, generated using GPT-4o and Claude-3-5-Sonnet within three multi-agent frameworks (Authoritarian, Market-Based, and Hierarchical). It reveals several consistent strategies and notable differences.

Across all models and frameworks, clarity and conciseness, step-by-step instructions, and the use of examples are consistently emphasised, reinforcing the effectiveness of some existing best practices. However, it is observed that GPT-3.5-Turbo is more responsive to prompts that are simple and concise, whereas the open-source SLMs rely more on examples and additional contextual information. This does not come as a surprise since GPT-3.5-Turbo is trained on vast amounts of data and able to generalise well with limited input. The complex architecture of GPT-3.5-Turbo enables it to leverage the inherent structure and semantics of language to understand concise prompts, whereas the simpler architecture of SLMs results in more literal interpretations, requiring explicit examples and context to understand the nuances of tasks.

Regarding agent models, GPT-4o tends to generate prompts that prioritise direct language and simplicity, while Claude-3-5-Sonnet is more likely to introduce creative approaches. Across all tasks, GPT-4o-generated prompts embody clarity and task alignment, whereas Claude-3-5-Sonnet-generated prompts are more adventurous encouraging visualisation or utilising immersive scenarios. The emphasis on creative approaches and visualisation by Claude-3-5-Sonnet was somewhat unexpected, particularly in contrast to GPT-4o's more straightforward and direct approach. This supports observations in earlier studies speculating that different models offer different perspectives on effective prompting strategies. It is hypothesised that these contrasting methodologies could be blended to improve current prompting

approaches.

The strategies employed in crafting prompts for different tasks reveal distinct requirements and methodologies tailored to the specific nature of each task. Analysing the nuances of task-specific prompts provides insight into how these strategies align with the objectives and constraints of each task. For SST-2, role assignment within prompts seemingly contributes to enhanced performance, guiding the model to identify and interpret sentiments accurately. The complexity of mathematical problem-solving necessitates a structured approach, where prompts are crafted to guide the model through each stage of the solution process. By incorporating detailed instructions and worked examples, prompts help the model navigate the problem-solving process effectively, ensuring that each step is methodically addressed. The HumanEval task, which focuses on code generation, requires a comprehensive approach involving detailed instructions, examples, and thorough documentation. The prompts typically include extensive context and documentation to assist the model in generating code effectively, thus ensuring that the process is both rigorous and reflective of the complex requirements of software engineering tasks.

The prompts generated by different frameworks also exhibit distinct characteristics, which appear to correlate with the inherent structures and operational principles of these frameworks. Specifically, prompts derived from the Authoritarian framework are marked by a high degree of procedural detail and rigid adherence to the task requirements. This framework's prompts are characterised by their systematic and prescriptive nature, ensuring that each step in the process is meticulously defined and aligned with the overarching objective. In contrast, prompts from the Market-Based framework are distinguished by their emphasis on contextual relevance, role assignment, and aspect-based analysis. The Market-Based framework's prompts often reflect the dynamic and competitive nature of the market environment in which they are generated, exhibiting lots of detail and prompting techniques. This characteristic can be understood through the framework's empowerment of individual agent specialities rather than enforcing a collective review of feedback. Each agent in the Market-Based framework operates with a degree of autonomy, making decisions based on its own perspective and objectives. As a result, the prompts frequently incorporate nuanced aspect-based analysis to address the various elements pertinent to each agent's role and context. Prompts originating from the Hierarchical framework, on the other hand, frequently exhibit a structured and incremental approach. This is reflective of the framework's hierarchical organisation, which inherently supports a step-by-step process. The prompts are also usually shorter, possibly due to distillation at each tier of the hierarchy which results in more conciseness. Overall, the varied nature of prompts across these frameworks underscores how the architectural design of each framework influences the generation of prompts. Understanding these differences can provide valuable

insights into optimising prompt generation techniques.

This study offers unique insights into how specific strategies impact performance across varied tasks. The detailed categorisation of models and frameworks helps in understanding the nuanced differences in prompt design strategies. However, some key limitations with the study include the limited number of conversation logs analysed, and the possible lack of freedom afforded to the agents in the frameworks. Since only a handful of conversation logs are reviewed per task for each model, it's difficult to establish concrete conclusions and prompt recommendations with confidence. In many cases only one of three prompts per framework out performed the baseline, resulting in a limited view on how the framework can produce high performing prompts. In addition, the core principles help with the directed refinement of the prompt and more efficient exploration of the prompt space, however, they also moderate the extent of the exploration making the discovery of new best practices less likely. Similarly, implementing prompt design agents based on previous research may provide useful guidance but it encourages reliance on existing approaches. Insights derived from this experiment could be used to inform more effective designs of the frameworks.

In conclusion the study demonstrates the variability in effectiveness of prompt techniques across different domains, which not only emphasises the importance of domain-adaptable automated prompt optimisations but offers insight into how the prompt engineering landscape can be improved. The differing multi-agent architectures result in different interactions between agents and hence unique prompt styles which vary in effectiveness between models and tasks. The results provide foundations to curate an effective prompt engineering guide as well as refine the design of multi-agent frameworks.

Chapter 6

Conclusion and Future Work

This chapter concludes the thesis and revisits the key findings of its studies, highlights the achievements and contributions, and provides recommendations for future work to be carried on by other researchers.

6.1 Summary

The significance of this research lies in its introduction of a domain-adaptable approach to automated prompt optimisation, which demonstrates robustness across a variety of models and tasks. This research investigates three distinct multi-agent frameworks, each characterised by a unique architecture, and assesses their capability to refine a simple baseline prompt to enhance the performance of various LLMs. Given that prompts can substantially influence the performance of LLMs, yet optimising them remains a challenging endeavor often plagued by ineffective prompters, this research offers a valuable method for generating high-quality prompts from initially suboptimal ones. The frameworks not only automate the improvement of prompts but do so in a directed manner, thereby exploring the expansive prompt space more efficiently. Achieving domain adaptability is crucial for producing highly specified prompts, which is accomplished by tailoring the roles and core principles of agents within the multi-agent frameworks.

The three studies presented in this research introduce the frameworks and provide empirical evidence of their effectiveness, highlighting the key actions that lead to high-performing prompts. The research begins with a formalisation of the frameworks and their architectures, followed by an exploration of their effectiveness across a range of models and tasks, and an exposition of the key strategies they employ. This chapter provides a summary of the insights and achievements of the study, and concludes with recommendations for future research directions.

Authoritarian, Market-Based and Hierarchical Optimisation Frameworks

The thesis begins with the formalisation of three novel multi-agent frameworks designed for automated prompt optimisation. These frameworks encompass three distinct architectures: Authoritarian, Market-Based, and Hierarchical, each utilising unique feedback mechanisms. The implementation of these frame-

works was carried out using both GPT-4o and Claude-3-5-Sonnet agents, with the aim of determining the effects of each. The generated prompts were subsequently evaluated using GPT-3.5-Turbo.

GPT-3.5-Turbo demonstrates high performance with simple prompts across mathematical reasoning, sentiment analysis, and code generation. As such, the improvements are limited when employing the framework-generated prompts. The GPT-4o Authoritarian and Claude-3-5-Sonnet Hierarchical frameworks yield improved average performance on the HumanEval dataset, yet they fall short of the performance achieved with ZS-CoT prompting. Conversely, the GPT-4o Hierarchical and Claude-3-5-Sonnet Authoritarian frameworks exhibit lower than Baseline average performance but achieve high-performing maximums, with the Claude-3-5-Sonnet Authoritarian framework matching the ZS-CoT performance.

A notable distinction was observed between the prompts generated by GPT-4o and those generated by Claude-3-5-Sonnet. The latter's prompts were consistently longer, exhibiting more structure and creativity. These prompts often included detailed guidance, which was not necessarily required for simple tasks. The findings indicate that while the frameworks offer minimal gains for simple tasks when the prompts are used with state-of-the-art third-party models, they show potential in reliably producing sophisticated domain-specific prompts. Further testing on more complex tasks is recommended to fully ascertain their efficacy for state-of-the-art models.

Enhancing Open-Source SLMs to Rival Third-Party Models

The prompts generated by the frameworks are subsequently evaluated using two open-source models that are significantly less powerful than GPT-3.5-Turbo: Mistral-V0.3-7B and Llama-3.1-8B. The results indicate that the frameworks are highly effective in producing prompts for use by weaker models, particularly for tasks involving mathematical reasoning and sentiment analysis. Specifically, the prompts led to improvements in accuracy of up to 30% and 60% on the GSM8K dataset for Mistral-V0.3-7B and Llama-3.1-8B, respectively. Similar gains were observed on the SST-2 dataset, where accuracy rates increased from 80% to match the 96.7% accuracy achieved by GPT-3.5-Turbo.

While the improvements are less pronounced on the HumanEval dataset, the Llama-3.1-8B model does achieve a maximum accuracy of 73.3% when using prompts generated by the GPT-4o Hierarchical framework, approaching the 80% accuracy achieved by GPT-3.5-Turbo with the baseline prompts. Despite the significant improvements, the prompts for GSM8K exhibit high variance in performance. This variability is often due to formatting issues and the failure to recognise correct answers during evaluation, a problem particularly prominent with prompts generated by Claude-3-5-Sonnet. This is considered both a flaw in the evaluation process and of the prompts themselves. Unlike GPT-4o, which concluded

prompts with output instructions, Claude-3-5-Sonnet placed output instructions mid-prompt, resulting in them being seemingly neglected.

The empirical evidence from this study suggests that the multi-agent frameworks are remarkably effective in optimising prompts for SLMs, thereby narrowing the performance gap with GPT-3.5-Turbo.

Agent Interactions for High-Performing Prompt Generation

The key features of high-performing prompts are identified, along with the interactions between agents that lead to their creation. This analysis provides detailed insights into the differing approaches between the frameworks, as well as between GPT-4o and Claude-3-5-Sonnet. Additionally, it reveals effective task-specific techniques.

Current best practices, including clarity, step-by-step instructions, and the use of examples, are frequently implemented by agents across all frameworks, regardless of the task. The creativity exhibited by Claude-3-5-Sonnet prompts is reiterated through more in-depth analysis, highlighting the use of visualisation and immersive scenarios. Each framework contributes a diverse set of effective techniques: the Authoritarian framework often results in direct prompts, the Market-Based framework emphasises contextual relevance, and the Hierarchical framework employs a clear simple structure. Task-specific concepts are also utilised; for example, self-review and role design are notably effective for code generation, while more general concepts are effective for GSM8K and SST-2.

The study offers intriguing insights into how the different dynamics within the multi-agent frameworks enforce unique approaches to prompt refinement, thereby demonstrating the variety of effective strategies available to prompters. The results provide a foundation for further experimentation, which could explicitly test and blend the strategies implemented by the frameworks.

6.2 Contributions

This thesis makes several contributions to the literature. Foremost, it introduces three novel frameworks for domain-adaptable automated prompt optimisation, utilising multi-agent architectures to perform directed refinement of prompts reliably. It provides a comprehensive evaluation of these frameworks across a diverse set of tasks and a range of models. Additionally, it offers an in-depth analysis of the interactions among agents during the prompt optimisation process, providing a unique perspective on prompt engineering. The study identifies best practices employed by the agents and highlights a selection of general, task-specific, and model-specific strategies.

The research also presents several contributions to industry. The frameworks help optimise the prompt generation process, improving the efficiency and effectiveness of LLM-driven operations. The frameworks can increase robustness by reducing errors through multi-agent validation and cross-checking, ensuring higher reliability and compliance. They also empower non-technical users to create expert-level prompts, increasing productivity and output quality. Additionally, the automation of prompt engineering streamlines processes for generative AI engineering teams, allowing them to focus on more technically demanding tasks and optimise resource use. Overall, the contributions improve both the usability and reliability of LLMs across various sectors.

The validation of this research is bolstered by collaboration with ZILO, a rapidly growing AI software company which provided guidance on technical approaches and experimental design.

6.3 Future Work

While the research offers valuable insights, it is not without its limitations. There are opportunities for improvement in both the frameworks and the experimental procedures. The studies suggest several potential extensions, and the following outlines future work to further advance the use of multi-agent frameworks in automated prompt optimisation.

Authoritarian, Market-Based and Hierarchical Optimisation Frameworks

A key extension of this research involves enhancing the frameworks themselves. Potential improvements include developing a fully autonomous end-to-end process with automated agent generation and core principle formulation. This would require accurately establishing the domain of the prompt and creating appropriate expert agents. Additionally, fine-tuning the existing prompt design agents, inspired by the insights of the final study, could be a valuable area of focus. The frameworks currently exhibit high variability in their outputs; therefore, an approach to increase the deterministic nature of the process may be beneficial to ensure consistent results. Furthermore, given the potential of these frameworks, it is advisable to test them on more complex tasks and evaluate their effectiveness when using state-of-the-art models.

Enhancing Open-Source SLMs to Rival Third-Party Models

A practical limitation identified in study two is its reliance on a specific output format. Developing a more effective evaluation framework with fuzzy matching would yield a more informative assessment of

prompt performance. Additionally, investigating why the Claude-3-5-Sonnet model tends to incorporate output instructions mid-prompt could provide valuable insights into optimising the model's utility as an agent. Conducting a structured evaluation across increasing model sizes would offer a broader perspective on the impact of the frameworks. Of particular interest would be the performance of the prompts with the largest and best open-source models compared to the best third-party models.

Agent Interactions for High-Performing Prompt Generation

Investigating the impact of increasing the autonomy of agents by limiting the guidance provided could be a compelling avenue for future research. Redesigning agents to enhance their capacity to explore the prompt space may reveal additional best practices in prompt engineering. Given sufficient financial resources, generating a significantly larger number of prompts to expand the set of conversation logs would yield a more exhaustive sample of agent dynamics, thereby enabling more conclusive analyses. Additionally, integrating the techniques employed by each framework and agent model and assessing their effectiveness could contribute valuable insights towards the development of standardised prompting methodologies.

Bibliography

- [1] L. C. Adams et al. “Llama 3 Challenges Proprietary State-of-the-Art Large Language Models in Radiology Board-style Examination Questions”. In: (2024).
- [2] G. Armano et al. “An Agent Architecture for Planning in a Dynamic Environment”. In: (2001), pp. 388–394. DOI: 10.1007/3-540-45411-X_41.
- [3] Emily M. Bender et al. “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?” In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (2021). DOI: 10.1145/3442188.3445922.
- [4] Yoshua Bengio et al. “A Neural Probabilistic Language Model”. In: *J. Mach. Learn. Res.* 3 (2003), pp. 1137–1155.
- [5] Maciej Besta et al. “Graph of Thoughts: Solving Elaborate Problems with Large Language Models”. In: *ArXiv abs/2308.09687* (2023). DOI: 10.48550/arXiv.2308.09687.
- [6] Susmitha Bezawada et al. “Word2Vec”. In: *Natural Language Engineering* 23 (2016), pp. 155–162. DOI: 10.1017/S1351324916000334.
- [7] M. Bommarito and D. Katz. “GPT Takes the Bar Exam”. In: *ArXiv abs/2212.14402* (2022). DOI: 10.48550/arXiv.2212.14402.
- [8] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *ArXiv abs/2005.14165* (2020).
- [9] Sondos Mahmoud Bsharat et al. “Principled Instructions Are All You Need for Questioning LLaMA-1/2, GPT-3.5/4”. In: (2024). arXiv: 2312.16171 [cs.CL]. URL: <https://arxiv.org/abs/2312.16171>.
- [10] L. Buşoniu et al. “Multi-agent Reinforcement Learning: An Overview”. In: (2010), pp. 183–221. DOI: 10.1007/978-3-642-14435-6_7.
- [11] Chi-Min Chan et al. “ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate”. In: *ArXiv abs/2308.07201* (2023). DOI: 10.48550/arXiv.2308.07201.
- [12] Weize Chen et al. “AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors in Agents”. In: *ArXiv abs/2308.10848* (2023). DOI: 10.48550/arXiv.2308.10848.
- [13] Yuyan Chen et al. “MAPO: Boosting Large Language Model Performance with Model-Adaptive Prompt Optimization”. In: (2023), pp. 3279–3304. DOI: 10.18653/v1/2023.findings-emnlp.215.
- [14] Aakanksha Chowdhery et al. “PaLM: Scaling Language Modeling with Pathways”. In: *J. Mach. Learn. Res.* 24 (2022), 240:1–240:113.
- [15] Jiayi Cui et al. “ChatLaw: Open-Source Legal Large Language Model with Integrated External Knowledge Bases”. In: *ArXiv abs/2306.16092* (2023). DOI: 10.48550/arXiv.2306.16092.

- [16] Mingkai Deng et al. “RLPrompt: Optimizing Discrete Text Prompts with Reinforcement Learning”. In: (2022), pp. 3369–3391. DOI: 10.48550/arXiv.2205.12548.
- [17] Jesse Dodge et al. “Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus”. In: (2021), pp. 1286–1305. DOI: 10.18653/v1/2021.emnlp-main.98.
- [18] A. Dorri et al. “Multi-Agent Systems: A Survey”. In: *IEEE Access* 6 (2018), pp. 28573–28593. DOI: 10.1109/ACCESS.2018.2831228.
- [19] Abhimanyu Dubey et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024). Available at <https://arxiv.org/abs/2407.21783>.
- [20] P. Ersoy and M. Erşahin. “Benchmarking Llama 3 70B for Code Generation: A Comprehensive Evaluation”. In: (2024).
- [21] Chrisantha Fernando et al. “Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution”. In: *ArXiv abs/2309.16797* (2023). DOI: 10.48550/arXiv.2309.16797.
- [22] Yao Fu et al. “Improving Language Model Negotiation with Self-Play and In-Context Learning from AI Feedback”. In: *ArXiv abs/2305.10142* (2023). DOI: 10.48550/arXiv.2305.10142.
- [23] Andrea Galassi et al. “Attention in Natural Language Processing”. In: *IEEE Transactions on Neural Networks and Learning Systems* 32 (2019), pp. 4291–4308. DOI: 10.1109/TNNLS.2020.3019893.
- [24] Qingyan Guo et al. “Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers”. In: *ArXiv abs/2309.08532* (2023). DOI: 10.48550/arXiv.2309.08532.
- [25] Jayesh K. Gupta et al. “Cooperative Multi-agent Control Using Deep Reinforcement Learning”. In: (2017), pp. 66–83. DOI: 10.1007/978-3-319-71682-4_5.
- [26] Sirui Hong et al. “MetaGPT: Meta Programming for Multi-Agent Collaborative Framework”. In: *ArXiv abs/2308.00352* (2023). DOI: 10.48550/arXiv.2308.00352.
- [27] Wenyue Hua et al. “War and Peace (WarAgent): Large Language Model-based Multi-Agent Simulation of World Wars”. In: *ArXiv abs/2311.17227* (2023). DOI: 10.48550/arXiv.2311.17227.
- [28] Lei Huang et al. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *ArXiv abs/2311.05232* (2023). DOI: 10.48550/arXiv.2311.05232.
- [29] Susmit Jha et al. “Dehallucinating Large Language Models Using Formal Methods Guided Iterative Prompting”. In: *2023 IEEE International Conference on Assured Autonomy (ICAA)* (2023), pp. 149–152. DOI: 10.1109/ICAA58325.2023.00029.
- [30] J. Kaplan et al. “Scaling Laws for Neural Language Models”. In: *ArXiv abs/2001.08361* (2020).

- [31] Christin Kirchhübel and Georgina Brown. “Intellectual property rights at the training, development and generation stages of Large Language Models”. In: *Proceedings of the Workshop on Legal and Ethical Issues in Human Language Technologies @ LREC-COLING 2024*. Ed. by Ingo Siegert and Khalid Choukri. Torino, Italia: ELRA and ICCL, May 2024, pp. 13–18. URL: <https://aclanthology.org/2024.legal-1.3>.
- [32] Manuel Kolp et al. “Multi-Agent Architectures as Organizational Structures”. In: *Autonomous Agents and Multi-Agent Systems* 13 (2006), pp. 3–25. DOI: 10.1007/s10458-006-5717-6.
- [33] Weize Kong et al. “PRewrite: Prompt Rewriting with Reinforcement Learning”. In: *arXiv* (2024). eprint: 2401.08189 (cs.AI). URL: <https://arxiv.org/abs/2401.08189>.
- [34] Cheng Li et al. “Large Language Models Understand and Can be Enhanced by Emotional Stimuli”. In: *ArXiv abs/2307.11760* (2023). DOI: 10.48550/arXiv.2307.11760.
- [35] Tian Liang et al. “Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate”. In: *ArXiv abs/2305.19118* (2023). DOI: 10.48550/arXiv.2305.19118.
- [36] Na Liu et al. “From LLM to Conversational Agent: A Memory Enhanced Architecture with Fine-Tuning of Large Language Models”. In: *arXiv* (2024). eprint: 2401.02777 (cs.CL). URL: <https://arxiv.org/abs/2401.02777>.
- [37] Vivian Liu and Lydia B. Chilton. “Design Guidelines for Prompt Engineering Text-to-Image Generative Models”. In: *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (2021). DOI: 10.1145/3491102.3501825.
- [38] Yang Liu et al. *Datasets for Large Language Models: A Comprehensive Survey*. 2024. arXiv: 2402.18041 [cs.CL]. URL: <https://arxiv.org/abs/2402.18041>.
- [39] Zijun Liu et al. “Dynamic LLM-Agent Network: An LLM-agent Collaboration Framework with Agent Team Optimization”. In: *ArXiv abs/2310.02170* (2023). DOI: 10.48550/arXiv.2310.02170.
- [40] Yilin Ma et al. “Portfolio optimization with return prediction using deep learning and machine learning”. In: *Expert Syst. Appl.* 165 (2021), p. 113973. DOI: 10.1016/j.eswa.2020.113973.
- [41] Aman Madaan et al. “Self-Refine: Iterative Refinement with Self-Feedback”. In: *ArXiv abs/2303.17651* (2023). DOI: 10.48550/arXiv.2303.17651.
- [42] Zhao Mandi et al. “RoCo: Dialectic Multi-Robot Collaboration with Large Language Models”. In: *ArXiv abs/2307.04738* (2023). DOI: 10.48550/arXiv.2307.04738.
- [43] Tomas Mikolov et al. “Recurrent neural network based language model”. In: (2010), pp. 1045–1048. DOI: 10.21437/Interspeech.2010-343.
- [44] T. Nadarzynski et al. “Acceptability of artificial intelligence (AI)-led chatbot services in healthcare: A mixed-methods study”. In: *Digital Health* 5 (2019). DOI: 10.1177/2055207619871808.

- [45] Y. Narsupalli et al. “Review-Feedback-Reason (ReFeR): A Novel Framework for NLG Evaluation and Reasoning”. In: (2024).
- [46] OpenAI. *GPT-4 Technical Report*. 2024. arXiv: 2303.08774 [cs.CL]. URL: <https://arxiv.org/abs/2303.08774>.
- [47] J. Paananen and V. Fortino. “An omics perspective on drug target discovery platforms”. In: *Briefings in Bioinformatics* 21 (2019), pp. 1937–1953. DOI: 10.1093/bib/bbz122.
- [48] J. Park et al. “Generative Agents: Interactive Simulacra of Human Behavior”. In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (2023). DOI: 10.1145/3586183.3606763.
- [49] S. Park and V. Sugumaran. “Designing multi-agent systems: a framework and application”. In: *Expert Syst. Appl.* 28 (2005), pp. 259–271. DOI: 10.1016/j.eswa.2004.10.006.
- [50] Savvas Petridis et al. *ConstitutionalExperts: Training a Mixture of Principle-based Prompts*. 2024. arXiv: 2403.04894 [cs.CL]. URL: <https://arxiv.org/abs/2403.04894>.
- [51] Reid Pryzant et al. “Automatic Prompt Optimization with "Gradient Descent" and Beam Search”. In: (2023), pp. 7957–7968. DOI: 10.48550/arXiv.2305.03495.
- [52] Xipeng Qiu et al. “Pre-trained models for natural language processing: A survey”. In: *Science China Technological Sciences* 63 (2020), pp. 1872–1897. DOI: 10.1007/s11431-020-1647-3.
- [53] Vipula Rawte et al. “Exploring the Relationship between LLM Hallucinations and Prompt Linguistic Nuances: Readability, Formality, and Concreteness”. In: *ArXiv abs/2309.11064* (2023). DOI: 10.48550/arXiv.2309.11064.
- [54] R. Rosenfeld. “Two decades of statistical language modeling: where do we go from here?” In: *Proceedings of the IEEE* 88 (2000), pp. 1270–1278. DOI: 10.1109/5.880083.
- [55] Pranab Sahoo et al. “A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications”. In: (2024). arXiv: 2402.07927 [cs.AI]. URL: <https://arxiv.org/abs/2402.07927>.
- [56] Neil Savage. “Drug discovery companies are customizing ChatGPT: here’s how”. In: *Nature Biotechnology* 41.5 (2023), pp. 585–586.
- [57] Emily Sheng et al. “Societal Biases in Language Generation: Progress and Challenges”. In: (2021), pp. 4275–4293. DOI: 10.18653/v1/2021.acl-long.330.
- [58] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *arXiv* (2023). eprint: 2303.11366 (cs.AI). URL: <https://arxiv.org/abs/2303.11366>.
- [59] P. Stone and M. Veloso. “Multiagent Systems: A Survey from a Machine Learning Perspective”. In: *Autonomous Robots* 8 (2000), pp. 345–383. DOI: 10.1023/A:1008942012299.

- [60] Hao Sun et al. “Query-Dependent Prompt Evaluation and Optimization with Offline Inverse RL”. In: *ArXiv abs/2309.06553* (2023). DOI: 10.48550/arXiv.2309.06553.
- [61] Yashar Talebirad and Amirhossein Nadiri. “Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents”. In: *ArXiv abs/2306.03314* (2023). DOI: 10.48550/arXiv.2306.03314.
- [62] Tian Tan et al. “Cooperative Deep Reinforcement Learning for Large-Scale Traffic Grid Signal Control”. In: *IEEE Transactions on Cybernetics* 50 (2020), pp. 2687–2700. DOI: 10.1109/TCYB.2019.2904742.
- [63] A.J. Thirunavukarasu et al. “Large language models in medicine”. In: *Nature Medicine* 29 (2023), pp. 1930–1940. DOI: 10.1038/s41591-023-02448-8.
- [64] Hugo Touvron et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *ArXiv abs/2302.13971* (2023). DOI: 10.48550/arXiv.2302.13971.
- [65] Ashish Vaswani et al. “Attention is All you Need”. In: *Neural Information Processing Systems*. 2017. URL: <https://api.semanticscholar.org/CorpusID:13756489>.
- [66] Ruochen Wang et al. “Mixture-of-Experts in Prompt Optimization”. In: (2024). URL: <https://openreview.net/forum?id=sDmjlpPhdB>.
- [67] Xinyuan Wang et al. “PromptAgent: Strategic Planning with Language Models Enables Expert-level Prompt Optimization”. In: *ArXiv abs/2310.16427* (2023). DOI: 10.48550/arXiv.2310.16427.
- [68] K. Wangsa et al. “A Systematic Review and Comprehensive Analysis of Pioneering AI Chatbot Models”. In: (2024).
- [69] Jason Wei et al. “Chain of Thought Prompting Elicits Reasoning in Large Language Models”. In: *ArXiv abs/2201.11903* (2022).
- [70] Yinliang Xu et al. “Multiagent-Based Reinforcement Learning for Optimal Reactive Power Dispatch”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 42 (2012), pp. 1742–1751. DOI: 10.1109/TSMCC.2012.2218596.
- [71] Zelai Xu et al. “Language Agents with Reinforcement Learning for Strategic Play in the Werewolf Game”. In: *ArXiv abs/2310.18940* (2023). DOI: 10.48550/arXiv.2310.18940.
- [72] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *ArXiv abs/2210.03629* (2022).
- [73] Shunyu Yao et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”. In: *ArXiv abs/2305.10601* (2023). DOI: 10.48550/arXiv.2305.10601.
- [74] J. Zamfirescu-Pereira et al. “Why Johnny Can’t Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts”. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (2023). DOI: 10.1145/3544548.3581388.

- [75] Yongchao Zhou et al. “Large Language Models Are Human-Level Prompt Engineers”. In: *ArXiv* abs/2211.01910 (2022). DOI: 10.48550/arXiv.2211.01910.
- [76] Daniel M. Ziegler et al. “Fine-Tuning Language Models from Human Preferences”. In: *ArXiv* abs/1909.08593 (2019).

Chapter A

Source Code

Source code for the project can be found in the GitHub repository:

<https://github.com/iwatzon/prompt-optimisation>.

Chapter B

Agents

B.1 Prompt Design

Agent	Core Principles
Lead Prompt Writer	1. Always pay close attention to prompt design details. 2. Always make informed decisions with regards to prompt design. 3. Always be open-minded to feedback regarding prompt design. 4. Always be critical when reviewing prompt design.
Style and Structure	1. Always structure prompts logically for the task. 2. Always use a style and tone in prompts that is appropriate for the task. 3. Always assign a role to the language model that is relevant to the task.
Conciseness and Clarity	1. Always write clear and concise prompts. 2. Always use simple and direct language in prompts. 3. Always avoid ambiguity in prompts.
Contextual Relevance	1. Always provide context to help the model understand the task. 2. Always write prompts informed by the context of the task. 3. Always design contextually relevant roles for the language model.

Agent	Core Principles
Task Alignment	1. Always write prompts that align with the task criteria. 2. Always tailor instructions to the task to guide the model. 3. Always make the task abundantly clear to the model in the prompt.
Example Demonstration	1. Always provide examples to help the model understand the task. 2. Always provide examples that cover a range of complexities. 3. Always demonstrate the expected output of the model.
Incremental Prompting	1. Always break down complex tasks. 2. Always write clear step-by-step instructions to guide the model. 3. Always write instructions appropriate for the task complexity.

B.2 HumanEval

Agent	Core Principles
Lead Software Engineer	1. Always pay close attention to tasks concerning software engineering. 2. Always make informed decisions with regards to software engineering. 3. Always be open-minded to feedback regarding software engineering. 4. Always be critical when reviewing software engineering outputs.
Software Engineer	1. Always follow best practices in software engineering. 2. Always write clean and maintainable code. 3. Always consider the performance implications of code.

Agent	Core Principles
Software Architecture	1. Always design software architecture that is scalable and maintainable. 2. Always consider the trade-offs of different architectural patterns. 3. Always document software architecture decisions.
Code Reviewer	1. Always review code for errors and inefficiencies. 2. Always consider edge cases in code. 3. Always provide constructive feedback in code reviews.

B.3 GSM8K

Agent	Core Principles
Lead Mathematician	1. Always pay close attention to tasks concerning mathematics. 2. Always make informed decisions with regards to mathematics. 3. Always be open-minded to feedback regarding mathematics. 4. Always be critical when reviewing mathematics calculations.
Mathematician	1. Always stop to think and develop a mathematically sound plan to solve problems. 2. Always make an initial estimate of the answer before solving the problem. 3. Always use mathematical operators (addition, subtraction, multiplication, division) correctly. 4. Always double-check mathematical calculations.

Agent	Core Principles
Word Problem Solver	1. Always read the problem slowly and carefully to identify what the problem is asking you to find. 2. Always list the given facts and unknown facts. 3. Always rewrite the problem and facts in a more organized manner. 4. Always consider multiple approaches to solving problems.

B.4 SST-2

Agent	Core Principles
Lead Sentiment Analyst	1. Always pay close attention to tasks concerning sentiment analysis. 2. Always make informed decisions with regards to sentiment analysis. 3. Always be open-minded to feedback regarding sentiment analysis. 4. Always be critical when reviewing sentiment analysis outputs.
Graded Sentiment Analyst	1. Always consider the nuances of sentiment in text. 2. Always consider the level of positivity or negativity in text. 3. Always grade the sentiment of text before making a decision.
Emotive Sentiment Analyst	1. Always consider the emotional impact of text. 2. Always consider the emotive language in text. 3. Always consider lexical choices in text.
Aspect-Based Sentiment Analyst	1. Always break down the text into aspects. 2. Always consider the sentiment of each aspect. 3. Always consider how aspects contribute to the overall sentiment of the text.

Chapter C

Task Prompts

C.1 Traditional Prompts

GSM8K

Baseline

```
1 Solve the following maths word problem:
2 {content}
3 Please output your answer at the end as ##<your answer (arabic numerals)>
```

Zero-Shot Chain-of-Thought

```
1 Solve the following maths word problem:
2 {content}
3 Let's think step by step.
4 Please output your answer at the end as ##<your answer (arabic numerals)>
```

Emotive

```
1 Solve the following maths word problem:
2 {content}
3 This is very important to my career.
4 Please output your answer at the end as ##<your answer (arabic numerals)>
```

SST-2

Baseline

```
1 Classify the sentiment of the following text as either positive or negative:
2 {content}
3 Please output your answer at the end as ##<your answer (No format restrictions)>
```

Zero-Shot Chain-of-Thought

```
1 Classify the sentiment of the following text as either positive or negative:
2 {content}
3 Let's think step by step.
4 Please output your answer at the end as ##<your answer (No format restrictions)
```

Emotive

```
1 Classify the sentiment of the following text as either positive or negative:
2 {content}
3 This is very important to my career.
4 Please output your answer at the end as ##<your answer (No format restrictions)
```

HumanEval

Baseline

```
1 Complete the following function based on its signature and docstring:
2 {content}
3 Please output your answer at the end as ```python\n<your answer>\n```
```

Zero-Shot Chain-of-Thought

```
1 Complete the following function based on its signature and docstring:
2 {content}
3 Let's think step by step.
4 Please output your answer at the end as ```python\n<your answer>\n```
```

Emotive

```
1 Complete the following function based on its signature and docstring:
2 {content}
3 This is very important to my career.
4 Please output your answer at the end as ```python\n<your answer>\n```
```

C.2 Claude-3-5-Sonnet Prompts

GSM8K

Authoritarian Framework

```
1 Assume the role of a Mathematical Detective solving a numerical mystery. Your mission
   is to unravel this math word problem:
2
3 {content}
4
5 Investigation steps:
6 1. Decode the central question
7 2. Gather evidence (facts and unknowns)
8 3. Sketch a mental crime scene (visualize the problem)
9 4. Hypothesize a solution range
10 5. Follow the numerical clues step-by-step
11 6. Consider unconventional investigative methods
12 7. Cross-examine your calculations
13 8. Ponder what the solution reveals about the mystery
14
15 Present your findings as ##<your answer (arabic numerals)> with no spaces.
16
17 Case Study:
18 Mystery: The Discounted Shirt Conundrum
19 A store sells shirts for $25 each. If they offer a 20% discount on purchases of 3 or
   more shirts, how much would 5 shirts cost?
20
21 Solution:
22 1. Central question: What's the cost of 5 shirts with the discount?
23 2. Evidence: $25 per shirt, 20% discount for 3+ shirts, buying 5 shirts
24 3. Mental image: Five shirts with price tags, a "20% off" sign nearby
25 4. Hypothesis: Cost will be less than $125 (5 * $25) due to discount
26 5. Following the clues:
27   a) Regular price: 5 * $25 = $125
28   b) Discount amount: 20% of $125 = 0.2 * $125 = $25
29   c) Discounted price: $125 - $25 = $100
30 6. Unconventional method: Calculate per-shirt discounted price
31   a) Discounted price per shirt: $25 - (20% of $25) = $25 - $5 = $20
32   b) Total for 5 shirts: 5 * $20 = $100
33 7. Cross-examination: Both methods yield $100
```


34 8. Revelation: The discount effectively reduces each shirt's price by \$5

35

36 ##100

1 Solve this maths word problem:

2 {content}

3

4 Steps:

5 1. Identify the main question and list given facts.

6 2. Make an initial estimate of the answer.

7 3. Develop a mathematically sound plan to solve the problem.

8 4. Solve step-by-step, using appropriate mathematical operators (addition, subtraction, multiplication, division).

9 5. Double-check all calculations and reasoning.

10

11 Output your answer as ##<your answer (arabic numerals)> with no spaces.

12

13 Example:

14 Problem: John has 5 apples and gives 2 away. How many left?

15 Estimate: About 3

16 Solution: $5 - 2 = 3$

17 Double-check: $3 + 2 = 5$ (original amount)

18 ##3

1 Solve this maths word problem:

2 {content}

3

4 Follow these steps:

5 1. Identify the main question and list key information, including units.

6 2. Estimate the answer using mathematical reasoning.

7 3. Outline your solution approach, considering multiple methods.

8 4. Solve step-by-step, using correct math operators and maintaining units.

9 5. Verify your calculations, compare with your estimate, and justify your solution.

10 6. Consider any creative or unconventional approaches that might simplify the solution.

11

12 Output your answer as ##<number> with no spaces, including units if applicable.

13

14 Example:

15 Problem: John has 5 apples and buys 3 more. How many apples does he have now?

16 Solution:

17 1. Question: Total apples John has. Given: 5 initial, 3 bought. Unit: apples.

18 2. Estimate: $5 + 3 \approx 8$ apples (using simple addition).

```
19 3. Approach: Direct addition of quantities.
20 4. Solution: 5 apples + 3 apples = 8 apples
21 5. Verified. Matches estimate. Justified by the additive property of equality.
22 6. Creative approach: Could use set theory, defining sets A={{John's initial apples}}
    and B={{bought apples}}, then find |A or B| = |A| + |B| = 5 + 3 = 8.
23 Answer: ##8
```

Market-Based Framework

```
1 Solve this math problem:
2 {content}
3
4 Show your work clearly. Include key steps and reasoning.
5
6 Example 1 (Simple arithmetic):
7 Problem: A train travels 120 km in 2 hours. What is its average speed?
8 Work:
9 1. Speed = Distance ÷ Time
10 2. Distance = 120 km
11 3. Time = 2 hours
12 4. Speed = 120 km ÷ 2 hours = 60 km/h
13 Answer: ##60
14
15 Example 2 (Algebra):
16 Problem: If  $3x + 7 = 22$ , what is the value of  $x$ ?
17 Work:
18 1. Subtract 7 from both sides:  $3x = 15$ 
19 2. Divide both sides by 3:  $x = 5$ 
20 Answer: ##5
21
22 Example 3 (Geometry - incomplete, finish it):
23 Problem: A rectangle has a length of 8 cm and a width of 5 cm. What is its area?
24 Work:
25 1. Area of rectangle = length × width
26 2. Length = 8 cm, Width = 5 cm
27 3. Area = 8 cm × 5 cm = 40 cm2
28 Answer: ##40
29
30 Example 4 (Percentages):
31 Problem: A shirt originally priced at $80 is on sale for 25% off. What is the sale
    price?
```

```

32 Work:
33 1. Calculate the discount: 25% of $80 =  $0.25 \times \$80 = \$20$ 
34 2. Subtract the discount from the original price:  $\$80 - \$20 = \$60$ 
35 Answer: ##60
36
37 Now solve the given problem, show your work, and provide the answer in the required
    format.
38
39 Answer: ##<answer in arabic numerals>

```

```

1 SOLVE:
2 {content}
3
4 INTERACTIVE PROBLEM-SOLVING JOURNEY:
5 1. Warm-up: What's 7-3? (Solve this first!)
6   ##4
7   Excellent! Now let's tackle word problems.
8
9 2. Problem Evolution:
10  a) If 5 pens cost $10, how much does 1 pen cost?
11     Solution:  $\$10 / 5 = \$2$ 
12     ##2
13
14  b) If 1 pen costs $2, how many pens can you buy with $7?
15     Solution:  $\$7 / \$2 = 3.5$  pens
16     ##3.5
17
18  c) You have $7 and pens cost $2 each. After buying as many as possible,
19     what percentage of a pen can you buy with the remainder?
20     Solution:
21     Pens bought: 3 (costing $6)
22     Remainder: $1
23     Percentage of a pen:  $(\$1 / \$2) * 100 = 50\%$ 
24     ##50
25
26 ASCII ART PROBLEM-SOLVING:
27 For mixture problems:
28   Solution A   Solution B   Mixture
29   30%          70%         40%
30   |            |           |
31   V            V           V
32   [====]       [=====]   [=====]

```

```
33      10L      20L      30L
34
35 MISTAKE SPOTLIGHT:
36 Problem: A shirt costs $80. It's on sale for 25% off. What's the sale price?
37 Incorrect: $80 - 25 = $55 (subtracting percentage as a whole number)
38 Correct: $80 - ($80 * 0.25) = $80 - $20 = $60
39 ##60
40
41 PROBLEM-SOLVING MINDSET:
42 /* Visualize the problem: Draw if needed */
43 /* Break complex problems into simpler steps */
44 /* Always check if your answer makes logical sense */
45
46 SOLUTION STEPS:
47 1. Read and analyze
48 2. Plan your approach
49 3. Calculate step-by-step
50 4. Verify your result
51 5. Present as: ##<answer>
52
53 OUTPUT:
54 ##<your answer (arabic numerals)>
```

```
1 Solve:
2
3 {content}
4
5 1. Estimate.
6 2. Model. Define variables.
7 3. Solve twice. Show steps.
8 4. Check units.
9 5. Verify.
10 6. List errors, alternatives.
11 7. Apply to real world.
12
13 Answer: ##<number>
```

Hierarchical Framework

```
1 Solve:
2 {content}
3
```

```
4 Show all work.
```

```
5
```

```
6 ##<youranswer(arabicnumerals)>
```

```
1 Solve this maths word problem:
```

```
2 {content}
```

```
3 Output answer: ##<your answer (arabic numerals)> with no spaces.
```

```
1 You're a Word Problem Wizard. Decipher and solve this math word problem:
```

```
2
```

```
3 {content}
```

```
4
```

```
5 1. Identify the key information and question.
```

```
6 2. Translate the words into a mathematical equation.
```

```
7 3. Solve the equation, showing your work briefly.
```

```
8 4. Interpret the result in the context of the original problem.
```

```
9
```

```
10 Conclude with ##<your answer (arabic numerals)> (no spaces).
```

HumanEval

Authoritarian Framework

```
1 You will complete a Python function based on its signature and docstring. Follow these
  steps, outputting your progress after each step:
```

```
2
```

```
3 ‘‘python
```

```
4 {content}
```

```
5 Step 1: Implement the basic functionality described in the docstring.
```

```
6
```

```
7 Step 2: Add input validation and error handling for edge cases.
```

```
8
```

```
9 Step 3: Optimize the function for performance where possible.
```

```
10
```

```
11 Step 4: Add inline comments to explain any complex logic.
```

```
12
```

```
13 Step 5: Write at least one unit test or test case for the function.
```

```
14
```

```
15 Step 6: Review your implementation. Ensure it follows PEP 8 guidelines and is clean and
  readable.
```

```
16
```

```
17 Step 7: Add a brief "commit message" style comment explaining your implementation.
```

```
18
19 After each step, output your progress as:
20
21 <your answer so far>
22 After completing all steps, provide your final implementation as:
23
24 <your answer>
```

```
1 Let's implement the following function step by step:
2
3 {content}
4 Follow these steps:
5
6 Analyze the function signature and docstring. Explain your understanding.
7 Outline a high-level approach for the implementation.
8 Implement the function body, showing your work incrementally.
9 Refine the implementation, considering edge cases and optimizations.
10 Add inline comments and finalize the function.
11 After each step, output your progress as:
12
13 # Step X:
14 <your work for this step>
15 For the final output, provide the complete function as:
16
17 <your answer>
```

```
1 As an experienced Python Developer working on a data processing pipeline, your task is
  to implement the following function:
2
3 {content}
4 Follow these steps to complete the implementation:
5
6 Analyze the function signature and docstring carefully.
7 Identify the key components and operations required.
8 Plan your approach, considering potential edge cases and efficiency.
9 Implement the function step by step:
10 a. Set up any necessary variables or data structures.
11 b. Implement the core logic of the function.
12 c. Handle potential edge cases.
13 d. Optimize for efficiency where possible.
14 Review your implementation to ensure it follows the docstring specifications and Python
  best practices.
```

```
15 Here are two examples demonstrating this process:
16
17 Example 1 (Simple):
18 Input:
19
20 def square_list(numbers: List[int]) -> List[int]:
21     """
22     Square each number in the input list.
23
24     Args:
25     numbers (List[int]): A list of integers
26
27     Returns:
28     List[int]: A new list with each number squared
29     """
30 Output:
31
32 def square_list(numbers: List[int]) -> List[int]:
33     """
34     Square each number in the input list.
35
36     Args:
37     numbers (List[int]): A list of integers
38
39     Returns:
40     List[int]: A new list with each number squared
41     """
42     return [num ** 2 for num in numbers]
43
44 Explanation:
45 I used a list comprehension for efficiency. This approach squares each number in a
46     single line of code, creating a new list without modifying the original. It handles
47     the edge case of an empty list automatically.
48
49 Example 2 (More complex):
50 Input:
51
52 def process_data(data: List[Dict[str, Union[int, str]]]) -> Dict[str, List[int]]:
53     """
54     Process a list of dictionaries, grouping numeric values by string keys.
55
56     Args:
57     data (List[Dict[str, Union[int, str]]]): A list of dictionaries with string keys
```

```
    and int or str values
55
56     Returns:
57     Dict[str, List[int]]: A dictionary with string keys and lists of integers as values
58     """
59 Output:
60
61 from collections import defaultdict
62
63 def process_data(data: List[Dict[str, Union[int, str]]]) -> Dict[str, List[int]]:
64     """
65     Process a list of dictionaries, grouping numeric values by string keys.
66
67     Args:
68     data (List[Dict[str, Union[int, str]]]): A list of dictionaries with string keys
        and int or str values
69
70     Returns:
71     Dict[str, List[int]]: A dictionary with string keys and lists of integers as values
72     """
73     result = defaultdict(list)
74     for item in data:
75         for key, value in item.items():
76             if isinstance(value, int):
77                 result[key].append(value)
78     return dict(result)
79
80 Explanation:
81 I used a defaultdict to efficiently group values. The function iterates through each
        dictionary in the input list, checking if values are integers before adding them to
        the result. This handles edge cases like missing keys or non-integer values. The
        final conversion to a regular dict ensures the return type matches the docstring.
82 Now, implement the given function following this pattern. Output your answer as:
83
84 <your answer>
85 After your implementation, briefly explain your approach and any key decisions you made.
```

Market-Based Framework

```
1 Complete this function:
2
3 {content}
```



```
4 Match the docstring. Use efficient code. Handle errors if needed. Add brief comments
   only when necessary.
```

```
5
```

```
6 Output your answer as:
```

```
7
```

```
8 <your answer>
```

```
1 Your mission is to complete the given function based on its signature and docstring.
   Follow these micro-tasks:
```

```
2
```

```
3 Function Analysis:
```

```
4 a) Decode the function signature.
```

```
5 b) Unravel the docstring mysteries.
```

```
6 c) Craft a list of key requirements.
```

```
7
```

```
8 Logic Visualization:
```

```
9 Sketch a simple ASCII flowchart representing the function's logic.
```

```
10
```

```
11 Implementation Blueprint:
```

```
12 a) Identify essential variables.
```

```
13 b) Map out the main algorithmic steps.
```

```
14 c) Anticipate and list potential edge cases.
```

```
15
```

```
16 Code Crafting:
```

```
17 a) Build the function incrementally, step-by-step.
```

```
18 b) Sprinkle enlightening comments throughout.
```

```
19 c) Ensure your code speaks fluent Pythonic.
```

```
20
```

```
21 Self-Reflection Mirror:
```

```
22 a) Cross-check against the initial requirements.
```

```
23 b) Verify the handling of sneaky edge cases.
```

```
24 c) Judge your code's readability and efficiency.
```

```
25
```

```
26 Rubber Duck Debugging:
```

```
27 Explain your implementation to an imaginary junior developer named "Quackers".
```

```
28
```

```
29 Polish and Perfect:
```

```
30 Refine your code based on insights from steps 5 and 6.
```

```
31
```

```
32 Now, complete this function:
```

```
33
```

```
34 {content}
```

```
35
36 Present your polished function implementation in this format:
37
38 <your answer>
```

```
1 Complete this function:
2
3 {content}
4 Implement it. Handle edge cases. Apply best practices.
5
6 Your answer:
7
8 <your answer>
9 Explain key design choices.
```

Hierarchical Framework

```
1 Complete the following function based on its signature and docstring:
2
3 {content}
4 Please output your answer at the end as ‘‘python
5 <your answer>
```

```
1 You are a Python Function Implementer. Your task:
2
3 Analyze and complete this function:
4
5 ‘‘python
6 {content}
7 Follow these steps:
8
9 Study the function signature and docstring carefully.
10 Implement the function body to match the signature exactly.
11 Ensure your code fulfills all docstring requirements.
12 Handle potential edge cases and errors.
13 Optimize for efficiency and readability.
14 Provide your implementation as:
15
16 <your answer>
```

```
1 Complete the following function based on its signature and docstring:
2
```

```
3 {content}
4 Please output your answer at the end as “‘python
5 <your answer>
```

SST-2

Authoritarian Framework

```
1 You're SentiBot, tasked with classifying sentiment for SocialPulse. Analyze this
  comment:
2
3 {content}
4
5 Approach the analysis as a debate:
6 1. Present the case for positive sentiment (2-3 key points).
7 2. Present the case for negative sentiment (2-3 key points).
8 3. Weigh the arguments and declare a winner (positive or negative).
9 4. State your confidence level in the decision (0-100%).
10
11 Remember: Your final output MUST be either ##positive or ##negative (no spaces).
```



```
1 As an experienced graded sentiment analyst, evaluate the emotional tone of this text:
  {content}
2
3 Consider the nuances and intensity of sentiment, grading the level of positivity or
  negativity before making your final classification.
4
5 Use this grading scale:
6 -3: Extremely negative
7 -2: Very negative
8 -1: Somewhat negative
9 0: Neutral
10 +1: Somewhat positive
11 +2: Very positive
12 +3: Extremely positive
13
14 First, assign a grade, then classify the overall sentiment based on your grading.
15
16 Example:
17 Text: "The service was terrible, but the food was absolutely amazing!"
18 Analysis: The text contains both negative (-2) and positive (+3) elements. The positive
  sentiment is stronger.
```

```
19 Grade: +1
20 Output: ##positive
21
22 Now, analyze the given text. Provide your graded assessment and final classification
    using the following format:
23 Grade: [Your grade]
24 ##positive
25 or
26 ##negative
```

```
1 You are an expert sentiment analyst for a social media monitoring company. Analyze the
    emotional tone of this user-generated content:
2 {content}
3
4 Follow these steps:
5 1. Identify key words and phrases indicating sentiment.
6 2. Assess the overall context.
7 3. Consider cultural or demographic factors influencing interpretation.
8 4. Weigh positive elements against negative elements.
9 5. Determine if the overall sentiment is more positive or negative.
10
11 Based on your analysis, classify the overall sentiment.
12
13 Output your final classification as:
14 ##positive
15 or
16 ##negative
```

Market-Based Framework

```
1 Classify sentiment: {content}
2
3 - List aspects: Aspect | Sentiment (0=neg, 1=pos) | Weight (0-1)
4 - Calculate: Sum(Sentiment * Weight) / Sum(Weights)
5 - If result > 0.5, positive; else, negative
6
7 Output: ##positive or ##negative
```

```
1 Analyze {content}
2
3 Rate emotion 1-10.
4
```

```
5 End with:
6 ##positive
7 or
8 ##negative
```

```
1 Analyze {content}:
2
3 - List key topics
4 - For each: rate (-2 to +2), explain, note importance (1-5)
5 - Show topic links
6 - Note sentiment shifts
7 - Compare similar topics
8 - Overall: average, consider links and shifts, state confidence
9 - Summarize key points
10
11 End: ##positive or ##negative
```

Hierarchical Framework

```
1 You are a skilled sentiment analysis expert. Your task is to classify the overall
  sentiment of a given text as either positive or negative. Here are some examples to
  guide you:
2
3 Example 1:
4 Text: "I absolutely love this new smartphone! It's fast, sleek, and has an amazing
  camera."
5 Classification: ##positive
6
7 Example 2:
8 Text: "The customer service was terrible. I waited for hours and still didn't get my
  issue resolved."
9 Classification: ##negative
10
11 Example 3:
12 Text: "Despite the rainy weather, I managed to have a productive day at work."
13 Classification: ##positive
14
15 Now, analyze the sentiment of the following text:
16
17 {content}
18
```

```
19 Consider the language, context, and emotional tone in your analysis. Focus on key
    phrases, adjectives, and the overall sentiment expressed.
```

```
20
```

```
21 Provide your classification in the following format:
```

```
22 ##positive
```

```
23 or
```

```
24 ##negative
```

```
1 Classify sentiment: {content}
```

```
2 Output: ##positive or ##negative
```

```
1 Analyze the sentiment of this text:
```

```
2
```

```
3 {content}
```

```
4
```

```
5 Is it positive or negative?
```

```
6 Output your answer as ##positive or ##negative
```

C.3 GPT-4o Prompts

GSM8K

Authoritarian Framework

```
1 Solve the following maths word problem: {content} Output the answer as ##<your answer
  (arabic numerals)>. For example, if the problem is 'If a train travels 60 miles in
  1 hour, how far will it travel in 3 hours?', the output should be ##180. Please
  solve the problem step-by-step before providing the final answer.
```

```
1 Solve the following math word problem: {content} Ensure to output the answer at the end
  in the format ##<your answer (arabic numerals)> with no spaces.
```

```
1 Solve the following math word problem: {content}
```

```
2 Provide the answer in the format ##<your answer (arabic numerals)>
```

Market-Based Framework

```
1 Solve the following math word problem:
```

```
2 {content}
```

```
3 Output your answer as ##<your answer (arabic numerals)> with no spaces. For example, if
  the answer is 42, write it as ##42. Show all steps clearly.
```

```
4
5 **Example 1:**
6 **Problem:** If a train travels 60 miles in 1.5 hours, what is its average speed in
   miles per hour?
7 **Solution:** Divide the total distance by the total time: 60 miles / 1.5 hours = 40
   miles per hour.
8 **Output:** ##40
9
10 **Example 2:**
11 **Problem:** A rectangle has a length of 10 meters and a width of 5 meters. What is its
   area in square meters?
12 **Solution:** Multiply the length by the width: 10 meters * 5 meters = 50 square meters.
13 **Output:** ##50
14
15 **Example 3:**
16 **Problem:** A store is having a 20% off sale. If an item originally costs $50, what is
   the sale price?
17 **Solution:** Calculate 20% of $50:  $0.20 * \$50 = \$10$ . Subtract this from the original
   price:  $\$50 - \$10 = \$40$ .
18 **Output:** ##40
```

```
1 Solve this math problem:
2
3 {content}
4
5 Identify known values, set up equations, and solve for the unknown.
6
7 **Example 1**:
8 A train travels 60 miles in 1.5 hours. What is its average speed in miles per hour?
9 - Distance = 60 miles, Time = 1.5 hours
10 - Speed = Distance / Time
11 - Speed =  $60 / 1.5 = 40$  miles per hour
12 - Final answer: ##40
13
14 **Example 2**:
15 A car rental company charges a flat fee of $50 plus $0.20 per mile. If a customer has
   $90, how many miles can they drive?
16 - Flat fee = $50, Cost per mile = $0.20, Budget = $90
17 -  $50 + 0.20 * \text{Miles} = 90$ 
18 -  $0.20 * \text{Miles} = 40$ 
19 - Miles =  $40 / 0.20 = 200$  miles
20 - Final answer: ##200
```

```
21
22 Output your final answer as ##<your answer (arabic numerals)> with no spaces.

1 Solve the following math problem step-by-step:
2 {content}
3
4 1. Identify key information and variables:
5   - Look for numbers, units, and relationships between variables.
6   - Write down any given equations or formulas.
7
8 2. Break the problem into smaller parts and solve each part:
9   - Divide the problem into manageable sections.
10  - Solve each section individually, ensuring to check your work for accuracy.
11
12 3. Label each step clearly:
13   - Use labels such as "Step 1," "Step 2," etc.
14   - Provide a brief description of what is being done in each step.
15
16 4. Combine results for the final answer:
17   - Ensure all intermediate results are correct.
18   - Combine the results logically to arrive at the final answer.
19
20 5. Output the final answer as ##<your answer (arabic numerals)> with no spaces.
```

Hierarchical Framework

```
1 Solve the following math word problem:
2 {content}
3 Output the answer at the end as ##<your answer (arabic numerals)>

1 Solve the following maths word problem:
2 {content}
3
4 1. Identify key information.
5 2. Perform necessary calculations.
6 3. Double-check your work.
7
8 Output your final answer as ##<your answer (arabic numerals)> with no spaces.

1 Solve the following maths word problem step-by-step:
2 {content}
3 Output the final answer as ##<your answer (arabic numerals)> with no spaces.
```


HumanEval

Authoritarian Framework

```
1 Complete the following function based on its signature and docstring provided in the
   placeholder '{content}'. The docstring contains important information about the
   function's purpose and expected behavior. Use this information to implement the
   function logic.
```

```
2
3 Please output your answer at the end as:
4 '''python
5 <your answer>
```

```
1 Complete the function below using its signature and docstring. Use the information
   provided to infer the function's logic.
```

```
2
3 Example 1:
4 def add(a: int, b: int) -> int:
5     """
6     Adds two integers together.
7     """
8     # Your code here
```

```
9
10 # Expected completion:
11 def add(a: int, b: int) -> int:
12     """
13     Adds two integers together.
14     """
15     return a + b
```

```
16 Example 2:
17 def greet(name: str) -> str:
18     """
19     Returns a greeting message for the given name.
20     """
21     # Your code here
```

```
22
23 # Expected completion:
24 def greet(name: str) -> str:
25     """
26     Returns a greeting message for the given name.
27     """
28     return f"Hello, {{name}}!"
```

29 Now, complete the following function:

30

31 {content}

32 Output your answer as:

33

34 <your answer>

1 Complete the following function based on its signature and docstring. Ensure it matches the signature and docstring.

2

3 Example:

4

5 def add(a, b):

6 """

7 Adds two numbers and returns the result.

8

9 Parameters:

10 a (int): The first number.

11 b (int): The second number.

12

13 Returns:

14 int: The sum of the two numbers.

15 """

16 return a + b

17 You are a developer tasked with implementing functions based on provided specifications.

18

19 Now, complete the following function:

20

21 {content}

22 Output your answer as:

23

24 <your answer>

Market-Based Framework

1 Complete the following function based on its signature and docstring. Ensure that the implementation:

2

3 Follows best practices in software engineering

4 Writes clean and maintainable code

5 Considers the performance implications of the code

6 {content}

```
7 Please output your answer at the end as ‘‘python
8 <your answer>
```

```
1 Complete the function based on its signature and docstring. Ensure the implementation
  is:
2 - Scalable
3 - Maintainable
4 - Performance-conscious
5 - Secure
6
7 Document architectural decisions and reasons for chosen approaches. Aim for a modular
  design.
8
9 Examples:
10 Function Signature and Docstring:
11 ‘‘python
12 def add(a: int, b: int) -> int:
13     """
14     Adds two integers and returns the result.
15     """
16 Expected Implementation:
17
18 def add(a: int, b: int) -> int:
19     """
20     Adds two integers and returns the result.
21     """
22     return a + b
23
24 # Example usage:
25 # result = add(2, 3)
26 # print(result) # Output: 5
27 Function Signature and Docstring:
28
29 def factorial(n: int) -> int:
30     """
31     Returns the factorial of a non-negative integer.
32     """
33 Expected Implementation:
34
35 def factorial(n: int) -> int:
36     """
37     Returns the factorial of a non-negative integer.
```

```
38 """
39     if n == 0:
40         return 1
41     else:
42         return n * factorial(n - 1)
43
44 # Example usage:
45 # result = factorial(5)
46 # print(result) # Output: 120
47 Now, complete the following function:
48
49 {content}
50 Output your answer as ‘‘python
51 <your answer>
```

```
1 Complete the following function based on its signature and docstring. Follow these
   steps to ensure clarity and correctness:
2
3 1. **Understand the Function Purpose:**
4     - Read the function signature and docstring carefully to understand what the
       function is supposed to do.
5
6 2. **Outline the Steps:**
7     - Outline the main steps needed to achieve the function’s purpose. This will help in
       organizing the code logically.
8
9 3. **Handle Exceptions:**
10    - Identify any potential exceptions that need to be handled and write the necessary
       code to handle them.
11
12 4. **Write the Main Logic:**
13    - Implement the main logic of the function using meaningful variable names and
       avoiding code duplication.
14
15 5. **Add Comments:**
16    - Write necessary comments to explain the code and make it easier to understand.
17
18 6. **Optimize for Performance:**
19    - Ensure that the code is optimized for performance.
20
21 Here is the function signature and docstring you need to complete:
22 ‘‘python
```

```
23 {content}
24 Example 1: Simple Function
25 def add(a: int, b: int) -> int:
26     """
27     Adds two integers and returns the result.
28     """
29     return a + b
30 Example 2: Function with Error Handling
31 def divide(a: int, b: int) -> float:
32     """
33     Divides the first integer by the second and returns the result.
34     Raises a ValueError if the second integer is zero.
35     """
36     if b == 0:
37         raise ValueError("Cannot divide by zero")
38     return a / b
39 Please output your answer at the end as:
40
41 <your answer>
42 '''.
```

Hierarchical Framework

```
1 As a Python developer, complete the following function based on its signature and
  docstring. Follow these steps:
2 1. Ensure your implementation is logically consistent.
3 2. Handle edge cases where applicable.
4 3. Adhere to best coding practices.
5
6 '''python
7 {content}
8 Output your answer at the end as '''python
9 <your answer>
```



```
1 Complete the function below based on its signature and docstring. Use the information
  provided in the docstring to infer the logic. Format your final output as follows:
2
3 '''python
4 <your answer>
5 Here is the function signature and docstring:
6
7 {content}
```

```
1 Complete the following function based on its signature and docstring:
2
3 {content}
4 Output your completed function as:
5
6 <your answer>
```

SST-2

Authoritarian Framework

```
1 Classify the sentiment of the following text as either positive or negative: {content}
   Output your answer as ##positive or ##negative with no spaces.
```

```
1 Classify the sentiment of the following text as either positive or negative: {content}
2 Output your answer as ##positive or ##negative.
```

```
1 Please classify the sentiment of the following text as either positive or negative,
   taking into account various elements such as tone, context, and content: {content}
   Ensure to output the answer at the end as ##positive or ##negative with no spaces.
```

Market-Based Framework

```
1 Classify the sentiment of the following text as either positive or negative,
   considering the nuances and context of the sentiment expressed. Follow these steps:
2
3 1. Identify and list the aspects mentioned in the text.
4 2. Evaluate the sentiment of each aspect, considering the context and nuances of the
   language used. For example, words like "okay" or "fine" might be neutral but can
   lean positive or negative based on context. Pay special attention to the intensity
   of sentiment words (e.g., "terrible" is stronger than "bad").
5 3. Grade the sentiment of each aspect as positive, negative, or neutral. If an aspect
   is neutral, consider whether it leans positive or negative based on context.
6 4. Determine how these aspects contribute to the overall sentiment by weighing the
   number and intensity of positive and negative sentiments. Note that strong negative
   sentiments might outweigh multiple mild positive sentiments. Use a scale to balance
   sentiments, for example:
7   - Strong Positive: +2
8   - Mild Positive: +1
9   - Neutral: 0
10  - Mild Negative: -1
11  - Strong Negative: -2
```

```
12 5. Consider the overall balance of sentiments before making the final decision. If the
    total score is positive, classify the sentiment as positive; if negative, classify
    it as negative.
13 6. Output your answer at the end as ##positive or ##negative with no spaces.
14
15 Example:
16 Text: "The food was delicious, but the service was terrible."
17 Aspects:
18 - Food: Positive (delicious) +2
19 - Service: Negative (terrible) -2
20 Overall Sentiment: ##negative (due to the strong negative sentiment of the service)
21
22 {content}
23
24 Please assess the level of positivity or negativity before making a decision.
25 Output your answer at the end as ##positive or ##negative with no spaces.
```

```
1 Classify the sentiment of the following text as either positive or negative. Evaluate
    key aspects, including emotional impact, emotive language, lexical choices,
    context, and overall tone, to determine the overall sentiment. Positive sentiment
    typically includes language that conveys happiness, satisfaction, or approval,
    while negative sentiment includes language that conveys sadness, dissatisfaction,
    or disapproval. Output your answer as ##positive or ##negative with no spaces:
    {content}
```

```
1 Classify the sentiment of the following text as either positive or negative. Follow
    these steps:
2
3 1. List all aspects mentioned.
4 2. For each aspect, consider the emotional impact, emotive language, and lexical
    choices.
5 3. Assign a sentiment score (1 to 5) to each aspect: 1 is very negative, 3 is neutral,
    and 5 is very positive.
6 4. Average the scores to determine the overall sentiment.
7 5. Output ##positive or ##negative with no spaces. Also, provide the overall sentiment
    score from 1 to 5.
8
9 Consider the emotional impact, emotive language, and lexical choices in the text.
10
11 {content}
```

Hierarchical Framework

```
1 You are a sentiment analysis tool. Classify the sentiment of the following text as  
   positive or negative: {content}
```

```
2
```

```
3 Output as ##positive or ##negative.
```

```
4
```

```
5 Example:
```

```
6 Text: "I love this product!"
```

```
7 Output: ##positive
```

```
1 Classify the sentiment of the following text as either positive or negative. Break down  
   the text into different aspects and rate each aspect's sentiment on a scale from -5  
   (very negative) to +5 (very positive). Consider the emotional impact, emotive  
   language, and lexical choices. Provide a brief justification for your overall  
   classification. Describe the overall emotional tone and identify key emotive words  
   or phrases, explaining their significance. Consider the context and interrelations  
   between aspects, and identify any potential biases or conflicting sentiments:  
   {content}
```

```
2
```

```
3 Please output your answer at the end as ##positive or ##negative with no spaces.
```

```
1 Classify the sentiment of the following text as either positive or negative. Break down  
   the text into different aspects, consider the sentiment of each aspect, and  
   evaluate how these aspects contribute to the overall sentiment. Consider the  
   emotional impact, nuances, levels of positivity or negativity, emotive language,  
   and lexical choices in the text before making your decision. Please output your  
   answer at the end as ##positive or ##negative with no spaces: {content}
```